

LAYER: 2_MesCore

This file is a merged representation of a subset of the codebase, containing files not matching ignore patterns, combined into a single document by Repomix.

<file_summary>

This section contains a summary of this file.

<purpose>

This file contains a packed representation of a subset of the repository's contents that is considered the most important context.

It is designed to be easily consumable by AI systems for analysis, code review, or other automated processes.

</purpose>

<file_format>

The content is organized as follows:

1. This summary section
2. Repository information
3. Directory structure
4. Repository files (if enabled)
5. Multiple file entries, each consisting of:
 - File path as an attribute
 - Full contents of the file

</file_format>

<usage_guidelines>

- This file should be treated as read-only. Any changes should be made to the original repository files, not this packed version.
- When processing this file, use the file path to distinguish between different files in the repository.
- Be aware that this file may contain sensitive information. Handle it with the same level of security as you would the original repository.

</usage_guidelines>

<notes>

- Some files may have been excluded based on .gitignore rules and Repomix's configuration
- Binary files are not included in this packed representation. Please refer to the Repository Structure section for a complete list of file paths, including binary files
- Files matching these patterns are excluded: `**/obj/**`, `**/bin/**`, `**/*.png`, `**/*.jpg`, `**/node_modules/**`, `**/*.Designer.cs`
- Files matching patterns in .gitignore are excluded
- Files matching default ignore patterns are excluded
- Files are sorted by Git change count (files with more changes are at the bottom)

</notes>

</file_summary>

<directory_structure>

```
Promatis.Net.MES.Configuration/Class1.cs
Promatis.Net.MES.Configuration/Promatis.Net.MES.Configuration.csproj
Promatis.Net.MES.Data/Configuration/TechnologicalOperationParameterBaseConfiguration.cs
Promatis.Net.MES.Data/Configuration/TechnologicalOperationUnitBaseConfiguration.cs
Promatis.Net.MES.Data/Configuration/UnitBaseConfiguration.cs
Promatis.Net.MES.Data/DbContext/MesApplicationDbContext.cs
Promatis.Net.MES.Data/Promatis.Net.MES.Data.csproj
Promatis.Net.MES.Data/Triggers/TechnologicalOperationHierarchyTrigger.cs
Promatis.Net.MES.Data/Triggers/UnitBaseHierarchyTrigger.cs
Promatis.Net.MES.Domain.Interface/Enums/CalculationMethod.cs
Promatis.Net.MES.Domain.Interface/Enums/UnitKind.cs
Promatis.Net.MES.Domain.Interface/Enums/UnitType.cs
Promatis.Net.MES.Domain.Interface/Interfaces/ITechnologicalOperation.cs
Promatis.Net.MES.Domain.Interface/Interfaces/ITechnologicalParameter.cs
Promatis.Net.MES.Domain.Interface/Promatis.Net.MES.Domain.Interface.csproj
Promatis.Net.MES.Domain/Promatis.Net.MES.Domain.csproj
Promatis.Net.MES.Domain/TechnologicalOperation/TechnologicalOperationBase.cs
Promatis.Net.MES.Domain/TechnologicalOperation/TechnologicalOperationBaseValidator.cs
Promatis.Net.MES.Domain/TechnologicalOperation/TechnologicalOperationHierarchyEngine.cs
Promatis.Net.MES.Domain/TechnologicalOperationParameter/TechnologicalOperationParameterBase.cs
Promatis.Net.MES.Domain/TechnologicalOperationParameter/
TechnologicalOperationParameterBaseValidator.cs
Promatis.Net.MES.Domain/TechnologicalOperationUnit/TechnologicalOperationUnitBase.cs
Promatis.Net.MES.Domain/TechnologicalOperationUnit/TechnologicalOperationUnitBaseValidator.cs
Promatis.Net.MES.Domain/TechnologicalParameter/TechnologicalParameterBase.cs
Promatis.Net.MES.Domain/TechnologicalParameter/TechnologicalParameterBaseValidator.cs
Promatis.Net.MES.Domain/TechnologicalParameterCalcMethod/TechnologicalParameterCalcMethodBase.cs
```

```

Promatis.Net.MES.Domain/TechnologicalParameterCalcMethod/
TechnologicalParameterCalcMethodBaseValidator.cs
Promatis.Net.MES.Domain/TechnologicalParameterValue/TechnologicalParameterValueBase.cs
Promatis.Net.MES.Domain/TechnologicalParameterValue/TechnologicalParameterValueBaseValidator.cs
Promatis.Net.MES.Domain/UnitBase/UnitBase.cs
Promatis.Net.MES.Domain/UnitBase/UnitBaseValidator.cs
Promatis.Net.MES.Domain/UnitBase/UnitHierarchyEngine.cs
Promatis.Net.MES.Domain/UnitOfMeasurement/UnitOfMeasurement.cs
Promatis.Net.MES.Domain/UnitOfMeasurement/UnitOfMeasurementValidator.cs
Promatis.Net.MES.Service/Promatis.Net.MES.Service.csproj
Promatis.Net.MES.Service/TechnologicalOperationParameterService/
ITechnologicalOperationParameterService.cs
Promatis.Net.MES.Service/TechnologicalOperationParameterService/
TechnologicalOperationParameterService.cs
Promatis.Net.MES.Service/TechnologicalOperationService/ITechnologicalOperationService.cs
Promatis.Net.MES.Service/TechnologicalOperationService/TechnologicalOperationService.cs
Promatis.Net.MES.Service/TechnologicalParameterCalcMethodService/
ITechnologicalParameterCalcMethodService.cs
Promatis.Net.MES.Service/TechnologicalParameterCalcMethodService/
TechnologicalParameterCalcMethodService.cs
Promatis.Net.MES.Service/TechnologicalParameterService/ITechnologicalParameterService.cs
Promatis.Net.MES.Service/TechnologicalParameterService/TechnologicalParameterService.cs
Promatis.Net.MES.Service/UnitOfMeasurementService/IUnitOfMeasurementService.cs
Promatis.Net.MES.Service/UnitOfMeasurementService/UnitOfMeasurementService.cs
Promatis.Net.MES.Service/UnitService/IUnitBaseService.cs
Promatis.Net.MES.Service/UnitService/UnitBaseService.cs
Promatis.Net.MES.Tests/AssemblyInfo.cs
Promatis.Net.MES.Tests/Promatis.Net.MES.Tests.csproj
Promatis.Net.MES.Tests/Service/UnitServiceStandaloneTests.cs
Promatis.Net.MES.Tests/UnitBase/AppDbContext.cs
Promatis.Net.MES.Tests/UnitBase/TestUnit.cs
Promatis.Net.MES.Tests/UnitBase/TestUnitValidator.cs
Promatis.Net.MES.Tests/UnitBase/UnitBaseArchitectureTests.cs
Promatis.Net.MES.UI/_Imports.razor
Promatis.Net.MES.UI/Pages/UnitOfMeasurements/Card/UnitOfMeasurementEditDialog.razor
Promatis.Net.MES.UI/Pages/UnitOfMeasurements/Card/UnitOfMeasurementEditDialog.razor.cs
Promatis.Net.MES.UI/Pages/UnitOfMeasurements/UnitOfMeasurementPage.razor
Promatis.Net.MES.UI/Pages/UnitOfMeasurements/UnitOfMeasurementPageContext.cs
Promatis.Net.MES.UI/Promatis.Net.MES.UI.csproj
Promatis.Net.MES.UI/UiModule.cs
Promatis.Net.MES.UI/wwwroot/exampleJsInterop.js
</directory_structure>

```

<files>

This section contains the contents of the repository's files.

```

<file path="Promatis.Net.MES.Configuration/Class1.cs">
namespace Promatis.Net.MES.Configuration
{
    public class Class1
    {
    }
}
</file>

```

```

<file path="Promatis.Net.MES.Configuration/Promatis.Net.MES.Configuration.csproj">
<Project Sdk="Microsoft.NET.Sdk">
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
    </PropertyGroup>
    <ItemGroup>
        <ProjectReference Include="..\..\Core\Configuration.Web\Promatis.Net.Configuration.Web.csproj" />
    </ItemGroup>
    <ItemGroup>
        <ProjectReference Include="..\Promatis.Net.MES.Service\Promatis.Net.MES.Service.csproj" />
    </ItemGroup>
</Project>
</file>

```

```

<file path="Promatis.Net.MES.Data/Configuration/
TechnologicalOperationParameterBaseConfiguration.cs">

```

```

using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;

namespace Promatis.Net.MES.Data;

public abstract class TechnologicalOperationParameterBaseConfiguration<T, TOperation, TParameter> :
    IEntityTypeConfiguration<T>
    where T : TechnologicalOperationParameterBase<TOperation, TParameter> // <--- A05D 5CD0CT< C0;CâAC08CR ?C
    where TOperation : DomainObject, ITechnologicalOperation
    where TParameter : DomainObject, ITechnologicalParameter
{
    public virtual void Configure(EntityTypeBuilder<T> builder)
    {
        // 1. B4=C,,:C ;DÄ=D'9 C,,=CD5C0A CD;D0 ?D 5CD>D$2D 0D"5C08D0 4D41C'8D >C$0C08D0 ?C @C <CTBD >C" =C >C
        builder.HasIndex(x => new { x.OperationId, x.ParameterId })
            .IsUnique();
    }

    // 2. B A0%B A0 At Bd B0 TU%' d"ÄDU" ,, D ?D 0C$;CT=C,,5 CD;D0 6öGB FVÆWFR•
    // A,,AC0;DäGC 5CÄ 7C ?C,,ADÄ 8Cr 2D'1Cä@C08, CTAC'8 D44C ;CT=C AC <C AC$0CtL (ISoftDeletable.Deleted
    // A,, A, 5D ;C, 1D'; CÄ0C4:Câ CCD0C'5C0 AC < C0@C,,2D07C =C0KC' BCTEC0>C'>C48Dt5D :C,,9 C00D 0CÄ5D$@ (P
    builder.HasQueryFilter(x =>
        x.DeletedAt == null &&
        EF.Property<DateTime?>(x.Parameter, "DeletedAt") == null);
    }
}
</file>

```

```

<file path="Promatis.Net.MES.Data/Configuration/TechnologicalOperationUnitBaseConfiguration.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;

namespace Promatis.Net.MES.Data;

/// <summary>
/// A0>C0DC,,3D4@C FC,,O CD;D0 1C 7D² AC$0Ct8D
/// </summary>
/// <typeparam name="T"></typeparam>
public abstract class TechnologicalOperationUnitBaseConfiguration<T, TOperation> :
    IEntityTypeConfiguration<T>
    where T : TechnologicalOperationUnitBase<TOperation> // <--- A05D 5CD0CT< D$8C0 >C05D 0Dd8C•
    where TOperation : DomainObject, ITechnologicalOperation
{
    public virtual void Configure(EntityTypeBuilder<T> builder)
    {
        // 1. B4=C,,:C ;DÄ=D'9 C,,=CD5C0A CD;D0 ?C @D² ?Cä;CT9 (A0@CT4CäBC$@C IC 5D" 4D41C'8D >C$0C08CR AC$0Ct5
        builder.HasIndex(x => new { x.OperationId, x.UnitId })
            .IsUnique();
    }

    // 2. B A0%B A0 At Bd B0 TU%' d"ÄDU" ,, D ?D 0C$;CT=C,,5 CD;D0 6öGB FVÆWFR•
    // BD8C'LD$@D45CÄ AC <D2 AC$0CtCDäID4N D CD"=CäAD$L, CTAC'8 Cä=C ?Cä<CTGCT=C :C : D44C ;CT=C00D0í
    // A,, A, 5D ;C, CCD0C'5C0> D 2D07C =C0>CR A C05C' >C >D CCD>C$0C08CR ...Væ-B'í
    // A,,AC0>C'LCtCCT< EF.Property CD;D0 1CT7Cä?C AC0>C4> CD>D BD4?C : D$5C05C$KCÄ00C$=D'< D 2Cä9D BC$0C
    builder.HasQueryFilter(x =>
        x.DeletedAt == null &&
        EF.Property<DateTime?>(x.Unit, "DeletedAt") == null);
    }
}
</file>

```

```

<file path="Promatis.Net.MES.Data/Configuration/UnitBaseConfiguration.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Promatis.Net.MES.Domain;

namespace Promatis.Net.MES.Data;

public class UnitBaseConfiguration : IEntityTypeConfiguration<UnitBase>
{
    public void Configure(EntityTypeBuilder<UnitBase> builder)
    {

```

```

        // A$:C'NDt0CT< D BD 0D$5C48Dâ E B 4C'O C$ACT9 C,,5D 0D EC,,8 UnitBaseD
        builder.UseTptMappingStrategy();D
D
        // A00D BD >C":C ?Cä;CT9D
        builder.Property(x => x.Kind).IsRequired();D
        builder.Property(x => x.Type).IsRequired();D
    }D
}
</file>

<file path="Promatis.Net.MES.Data/DbContext/MesApplicationDbContext.cs">
using Microsoft.EntityFrameworkCore;D
using Microsoft.Extensions.Configuration;D
using Promatis.Net.Data;D
using Promatis.Net.MES.Domain;D
D
namespace Promatis.Net.MES.Data;D
D
public class MesApplicationDbContext(D
    DbContextOptions options,D
    IConfiguration configuration,D
    IServiceProvider? serviceProvider = null) D
    : ApplicationDbContext(options, configuration, serviceProvider)D
{D
    public DbSet<UnitOfMeasurement> UnitOfMeasurements => Set<UnitOfMeasurement>();D
}
</file>

<file path="Promatis.Net.MES.Data/Promatis.Net.MES.Data.csproj">
<Project Sdk="Microsoft.NET.Sdk">D
D
    <PropertyGroup>D
        <TargetFramework>net10.0</TargetFramework>D
        <ImplicitUsings>enable</ImplicitUsings>D
        <Nullable>enable</Nullable>D
    </PropertyGroup>D
D
    <ItemGroup>D
        <PackageReference Include="Microsoft.EntityFrameworkCore" Version="10.0.8" />D
        <PackageReference Include="Microsoft.EntityFrameworkCore.Relational" Version="10.0.8" />D
        <PackageReference Include="Microsoft.Extensions.Configuration" Version="10.0.8" />D
    </ItemGroup>D
D
    <ItemGroup>D
        <ProjectReference Include="..\..\Core\Data\Promatis.Net.Data.csproj" />D
        <ProjectReference Include="..\Promatis.Net.MES.Domain\Promatis.Net.MES.Domain.csproj" />D
    </ItemGroup>D
D
</Project>
</file>

<file path="Promatis.Net.MES.Data/Triggers/TechnologicalOperationHierarchyTrigger.cs">
using Microsoft.EntityFrameworkCore;D
using Promatis.Net.Data;D
using Promatis.Net.MES.Domain;D
D
namespace Promatis.Net.MES.Data;D
D
/// <summary>D
/// A 1D BD 0C=BC0KC' 1C 7Cä2D'9 D$@C,,3C45D 8CT@C @DT8C, BCTEC0>C'>C48Dt5D :C,,E Cä?CT@C FC,,9.D
/// At0D"8D"0CTB D BD CC=BD4@D2 >D" =CT:Cä@D 5C=BC0>C4> CD>C 0C$;CT=C,,O C0>CDCct;Cä2 C, :Cä=D$@Cä;C,,@D45D" AD
/// </summary>D
public abstract class TechnologicalOperationHierarchyTrigger<T, TOLink, TPLink> :
IBeforeSaveTrigger<T>D
    where T : TechnologicalOperationBase<T, TOLink, TPLink>D
    where TOLink : classD
    where TPLink : classD
{D
    public async Task HandleAsync(EntityCancelEventArgs<T> args)D
    {D
        T operation = args.Entity;D
D
        // ATAC'8 Cä?CT@C FC,,O C= >D =CT2C 0 – C0@Cä?D4AC=0CT< C0@Cä2CT@C=C0D
        if (!operation.ParentId.HasValue || operation.ParentId == Guid.Empty) return;D
D
        // A,,7C$;CT:C 5CÄ 8Cr !B4 AB AD$0D$CD –4ÆV b 8 Name D >CD8D$5C'LD :Cä9 Cä?CT@C FC,,8D

```

```

var parentInfo = await args.Context.Set<T>()
    .Where(x => x.Id == operation.ParentId)
    .Select(x => new { x.IsLeaf, x.Name })
    .FirstOrDefaultAsync();

if (parentInfo != null)
{
    // A,,ACô>C`LCtCCT< C00D, 4Cä<CT=C0KC' 4C$8Cd>C 4C`O C0@Cä2CT@C=8 C0@C 2C,,; C$;Cä6CT=C0>D BC,i
    // A05D 5CD0CT< D BC BD4A IsLeaf D >CD8D$5C`O, C$KD$0D"5C0=D`9 C,,7 C 0CtK.D
    if (parentInfo.IsLeaf)
    {
        args.Cancel = true;
        args.ErrorMessage = $"A00D CD,,5C08CR 8CT@C @DT8C, 0U3  "CTEC0>C`>C48Dt5D :C O Cä?CT@C FC,,O '{
            $"C05 CÄ>Cd5D" 1D`BDÄ 2C`>Cd5C00 C" w. &VçD-æf0ää ÖW0rÄ BC : C=0C  >C00
        };
        return;
    }
}

// AD>C0>C`=C,,BCT;DÄ=C O Ct0D"8D$0 C00 C,,7CÄ5C05C08CS 5D ;C, >C05D 0Dd8D0 AD$0C0>C$8D$AD0 C,,AD$>CÄÄ
// C0@Cä2CT@D05CÄ 2 ChangeTracker C,,;C, ABÄ =CTB C`8 D2 =CT5 CD>Dt5D =C,,E D0;CT<CT=D$>C-
if (operation.IsLeaf)
{
    bool hasChildrenInDb = await args.Context.Set<T>()
        .AnyAsync(x => x.ParentId == operation.Id);

    if (!TechnologicalOperationHierarchyEngine.CanChangeLeafStatus(hasChildrenInDb,
        operation.IsLeaf))
    {
        args.Cancel = true;
        args.ErrorMessage = $"A00D CD,,5C08CR FCT;CäAD$=CäAD$8 CD0C0=D`E: A05C$>Ct<Cä6C0> D 4CT;C BDÄ
            $"D$0C  :C : C$=D4BD 8 C05CR CCd5 D >CD5D 6C BD O CD@D43C,,5 D$5DT=Cä;Cä3C
        };
    }
}
}
</file>

```

```

<file path="Promatis.Net.MES.Data/Triggers/UnitBaseHierarchyTrigger.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Data;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;

namespace Promatis.Net.MES.Data;

/// <summary>
/// B$@C,,3C45D 4C`O C0@Cä2CT@C=8 D ?CTFC,,DC,,GC0KDR ?D 0C$8C^ 8CT@C @DT8C, Væ-D& 6Ri
/// </summary>
public class UnitBaseHierarchyTrigger : IBeforeSaveTrigger<UnitBase>
{
    public async Task HandleAsync(EntityCancelEventArgs<UnitBase> args)
    {
        UnitBase unit = args.Entity;

        if (!unit.ParentId.HasValue || unit.ParentId == Guid.Empty) return;

        // A0>C`CDt0CT< Kind C, æ ÖR @Cä4C,,BCT;D0 4C`O C,,=DD>D <C BC,,2C0>C4> D >Cä1D"5C08Dý
        var parentInfo = await args.Context.Set<UnitBase>()
            .Where(x => x.Id == unit.ParentId)
            .Select(x => new { x.Kind, x.Name })
            .FirstOrDefaultAsync();

        if (parentInfo != null && !UnitHierarchyEngine.IsHierarchyValid(parentInfo.Kind, unit.Kind))
        {
            args.Cancel = true;
            args.ErrorMessage = $"A00D CD,,5C08CR 8CT@C @DT8C, 0U3 C JCT:D" w.Væ-Bäæ ÖW0r ‡.Væ-Bä¶-æG0' " ½
                $"C05 CÄ>Cd5D" 1D`BDÄ 2C`>Cd5C0 2 '{parentInfo.Name}' ({parentInfo.Kind}).";
        }
    }
}
</file>

```

```

<file path="Promatis.Net.MES.Domain.Interface/Enums/CalculationMethod.cs">
using System.ComponentModel;

namespace Promatis.Net.MES.Domain.Interface;

```

```

Ð
/// <summary>Ð
/// AÄ5D$>CDK D 0D GCTBC BCTE. C00D 0CÄ5D$@Cä2Ð
/// </summary>Ð
[Flags]Ð
public enum CalculationMethodÐ
{Ð
    [Description("A05 Ct0CD0C0"•Ý
    None = 0, // 00000000Ð
    Ð
    [Description("AÄ0C=AC,,<C ;DÄ=Cä5 Ct=C GCT=C,,5 (MAX)")]Ð
    Max = 1,Ð
    Ð
    [Description("AÄ8C08CÄ0C´LC0>CR 7C00Dt5C08CR „0”ä”"•Ý
    Min = 2,Ð
    Ð
    [Description("B @CT4C05CR 7C00Dt5C08CR „ dr”"•Ý
    Avg = 4,Ð
    Ð
    [Description("A05D 2Cä5 Ct=C GCT=C,,5 (FIRST)")]Ð
    First = 8,Ð
    Ð
    [Description("A0>D ;CT4C05CR 7C00Dt5C08CR „Ä 5B”"•Ý
    Last = 16,Ð
    Ð
    [Description("A$ACR 7C00Dt5C08D0 „ ÄÄ”"•Ý
    All = 32Ð
}
</file>

<file path="Promatis.Net.MES.Domain.Interface/Enums/UnitKind.cs">
using System.ComponentModel;Ð
Ð
namespace Promatis.Net.MES.Domain.Interface;Ð
Ð
public enum UnitKindÐ
{Ð
    [Description("B :C´0CDAC=0D0 ;Cä3C,,AD$8C=0")]Ð
    Storage = UnitType.Storage | UnitType.Zone | UnitType.Rack | UnitType.Cell | UnitType.Crane,Ð
    Ð
    [Description("A0@Cä8Ct2Cä4D BC$5C0=C 0 Ct>C00")]Ð
    Production = UnitType.Workshop | UnitType.Section | UnitType.Line | UnitType.Workstation |
UnitType.MachineTool | UnitType.Table,Ð
    Ð
    [Description("B$@C =D ?Cä@D$=D´9 D47CT;")]Ð
    Transport = UnitType.Vehicle | UnitType.Conveyor,Ð
    Ð
    [Description("A0>CD@C 7CD5C´5C08CR"•Ý
    Department = UnitType.Workshop | UnitType.Section | UnitType.Other,Ð
    Ð
    [Description("B 0C >Dt0D0 BCäGC=0 / B0GCT9C=0")]Ð
    Position = UnitType.Cell | UnitType.OtherÐ
}
</file>

<file path="Promatis.Net.MES.Domain.Interface/Enums/UnitType.cs">
using System.ComponentModel;Ð
Ð
namespace Promatis.Net.MES.Domain.Interface;Ð
Ð
[Flags]Ð
public enum UnitTypeÐ
{Ð
    None = 0,Ð
    Ð
    [Description("Bd5DR"•Ý
    Workshop = 1,Ð
    Ð
    [Description("B4GC AD$>CΦ"•Ý
    Section = 2,Ð
    Ð
    [Description("A´8C08D0 0 Cä=C$5C”5D "•Ý
    Line = 4,Ð
    Ð
    [Description("B 0C >Dt5CR <CTAD$>")]Ð
    Workstation = 8,Ð

```

```

    [Description("B :C'0CB"•Ý
Storage = 16,Ð
    [Description("At>C00 DT@C =CT=C,,0")]Ð
Zone = 32,Ð
    [Description("B BCT;C'0Cb"•Ý
Rack = 64,Ð
    [Description("B0GCT9C@0 C 4D 5D 0 DT@C =CT=C,,0")]Ð
Cell = 128,Ð
    [Description("A@C = / A0>CDJCT<C08C#"•Ý
Crane = 256,Ð
    [Description("B BC =Cä:")Ð
MachineTool = 512,Ð
    [Description("A$5D AD$0C# 0 !D$>C²"•Ý
Table = 1024,Ð
    [Description("B$@C =D ?Cä@D$=Cä5 D @CT4D BC$>")]Ð
Vehicle = 2048,Ð
    [Description("A 2D$>C0>CÄ=D´9 D$@C =D ?Cä@D$5D "•Ý
Conveyor = 4096,Ð
    [Description("A0@CäGCT5")]Ð
Other = 8192Ð
}
</file>

<file path="Promatis.Net.MES.Domain.Interface/Interfaces/ITechnologicalOperation.cs">
namespace Promatis.Net.MES.Domain.Interface;Ð
Ð
public interface ITechnologicalOperationÐ
{Ð
    Ð
}
</file>

<file path="Promatis.Net.MES.Domain.Interface/Interfaces/ITechnologicalParameter.cs">
namespace Promatis.Net.MES.Domain.Interface;Ð
Ð
public interface ITechnologicalParameterÐ
{Ð
    Ð
}
</file>

<file path="Promatis.Net.MES.Domain.Interface/Promatis.Net.MES.Domain.Interface.csproj">
<Project Sdk="Microsoft.NET.Sdk">Ð
Ð
    <PropertyGroup>Ð
        <TargetFramework>net10.0</TargetFramework>Ð
        <ImplicitUsings>enable</ImplicitUsings>Ð
        <Nullable>enable</Nullable>Ð
    </PropertyGroup>Ð
Ð
    <ItemGroup>Ð
        <ProjectReference Include="..\..\Core\Domain.Interface\Promatis.Net.Domain.Interface.csproj" />Ð
    </ItemGroup>Ð
Ð
</Project>
</file>

<file path="Promatis.Net.MES.Domain/Promatis.Net.MES.Domain.csproj">
<Project Sdk="Microsoft.NET.Sdk">Ð
Ð
    <PropertyGroup>Ð
        <TargetFramework>net10.0</TargetFramework>Ð
        <ImplicitUsings>enable</ImplicitUsings>Ð
        <Nullable>enable</Nullable>Ð
    </PropertyGroup>Ð
Ð

```

```

<ItemGroup>ð
    <ProjectReference Include="..\..\Core\Domain\Promatis.Net.Domain.csproj" />ð
    <ProjectReference Include="..\
\Promatis.Net.MES.Domain.Interface\Promatis.Net.MES.Domain.Interface.csproj" />ð
</ItemGroup>ð
ð
</Project>
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalOperation/TechnologicalOperationBase.cs">
using Promatis.Net.Domain;ð
using Promatis.Net.Domain.Interface;ð
using Promatis.Net.MES.Domain.Interface;ð
ð
namespace Promatis.Net.MES.Domain;ð
ð
/// <summary>ð
/// A 0Ct>C$0Dð BCTEC0>C´>C48Dt5D :C 0 Cä?CT@C FC,,0.ð
/// A00D ;CT4D45D" GC,,AD$>CR =CR04Cd5C05D 8C¢ 4CT@CT2Câ !B4 ABÂ CC 8D 0Dð :Cä=DD;C,,:D$K CÄ0C0?C,,=C40,ð
/// C, ?D 5CD>D BC 2C´OCTB D BD >C4> D$8C08Ct8D >C$0C0=D´9 C,,=D$5D DCT9D 4C´O C 8Ct=CTA-C´>C48C¤8.ð
/// </summary>ð
public abstract class TechnologicalOperationBase<T, TOLink, TPLink> : ReferenceTreeBase<T>, // <- A¤ A,, "A,, 'AT
    ITechnologicalOperationð
    where T : TechnologicalOperationBase<T, TOLink, TPLink>ð
    where TOLink : classð
    where TPLink : classð
{ð
    public bool IsLeaf { get; set; } = true;ð
ð
    public virtual ICollection<TOLink> UnitLinks { get; set; } = new List<TOLink>();ð
    public virtual ICollection<TPLink> ParameterLinks { get; set; } = new List<TPLink>();ð
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalOperation/TechnologicalOperationBaseValidator.cs">
using FluentValidation;ð
using Promatis.Net.Domain;ð
using Promatis.Net.Domain.Interface;ð
using Promatis.Net.MES.Domain.Interface;ð
ð
namespace Promatis.Net.MES.Domain;ð
ð
// A$0C´8CD0D$>D >C05D 0Dd8C, ,,>D BC 5D$ADð 1CT7 C,,7CÄ5C05C08C´•
public abstract class TechnologicalOperationBaseValidator<T> : ReferenceTreeBaseValidator<T>ð
    where T : ReferenceTreeBase<T>, ITreeNode<T>, ITechnologicalOperation // <- A¤ A,, "A,, 'AT!A¤ AR B B A$ A
{ð
    protected TechnologicalOperationBaseValidator() : base()ð
    {ð
        // A 0Ct>C$KC´ &Vfw&Væ6T& 6Uf A-F F÷" CCd5 C0@Cä2CT@C,,; Cä1D"8CR ?Cä;Dð ,,BÂ æ ÖR•
        // At4CTADÂ <D² ?C,,HCT< C0@C 2C,,;C AD$@Cä3Cä 4C´O D$5DT=Cä;Cä3C,,GCTAC¤8DR >C05D 0Dd8C•
ð
        RuleFor(x => x.Code)ð
            .NotEmpty()ð
            .WithMessage("A¤>CB >C05D 0Dd8C, >C 0Ct0D$5C´5C0â"•
            .MaxLength(50)ð
            .WithMessage("A¤>CB >C05D 0Dd8C, =CR 4Cä;Cd5C0 ?D 5C$KD,,0D$L 50 D 8CÄ2Cä;Cä2.");ð
    }ð
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalOperation/
TechnologicalOperationHierarchyEngine.cs">
using System.Reflection;ð
using Promatis.Net.Domain.Interface;ð
ð
namespace Promatis.Net.MES.Domain;ð
ð
public static class TechnologicalOperationHierarchyEngineð
{ð
    /// <summary>ð
    /// A4;Cä1C ;DÄ=D´9 C,,AD$>Dt=C,,: C0@C 2CDK: C0@Cä2CT@D05D"Â 2C ;C,,4C0> C´8 C$;Cä6CT=C,,5 CD>Dt5D =CT9 Cä?C
    /// A00CÄ5D BC$> C ;Cä:C,,@D45D" ACä7CD0C08CR 2CTBCä: C$=D4BD 8 D$5D <C,,=C ;DÄ=D´E Cä?CT@C FC,,9 (IsLeaf =
    /// B$8C08Ct0Dd8Dð ?CT@CT2CT4CT=C =C GC,,AD$KC´ :Cä=D$@C :D" 4CT@CT2C BÄ GD$> C0>C´=CäAD$LDâ CD BD 0C00
    /// </summary>ð
    /// <typeparam name="T">A¤>C0:D 5D$=D´9 D$8C0 BCTEC0>C´>C48Dt5D :Cä9 Cä?CT@C FC,,8</typeparam>ð
    /// <param name="parent">B >CD8D$5C´LD :C 0 Cä?CT@C FC,,0</param>ð

```



```

/// <param name="childIsLeaf">Aô@C,,7C00C¢ BCä3CäÂ OC$;Dô5D$ADò ;C, ACä7CD0C$0CT<D´9/Cô5D 5CÄ5D"0CT<D´9 D
public static bool IsHierarchyValid<T>(T? parent, bool childIsLeaf)ð
    where T : class, ITreeNode<T>ð
{ð
    // 1. ATAC´8 D >CD8D$5C´O C05D"Â <D² ACä7CD0CT< C¤>D =CT2Cä9 DÔ;CT<CT=D" r MD$> C$ACT3CD0 C$0C´8CD=Cä
    if (parent == null)ð
    {ð
        return true;ð
    }ð
ð
    // 2. Bt5D 5Cr @CTDC´5C¤AC,,N (C,,;C, :C AD" : C,,=D$5D DCT9D C, CTAC´8 C K Cä= C KC²´ 8Ct2C´5C¤0CT< C0@
    // B$0C¢ :C : T C00D ;CT4D45D$ADò >D" FV6†æöÆöv-6 Ä÷ W& F-öä& 6RÄ AC$>C"AD$2Cä -4ÆV b BC < C40D 0C0BC
    PropertyInfo? isLeafProperty = typeof(T).GetProperty("IsLeaf");ð
    if (isLeafProperty != null)ð
    {ð
        bool parentIsLeaf = (bool)isLeafProperty.GetValue(parent)!;ð
        if (parentIsLeaf)ð
        {ð
            return false; // B$5D <C,,=C ;DÄ=D´9 D47CT; C05 CÄ>Cd5D" 1D´BDÄ @Cä4C,,BCT;CT<ð
        }ð
    }ð
ð
    return true;ð
}ð
ð
/// <summary>ð
/// Aô@Cä2CT@Dô5D"Â @C 7D 5D,,5C0> C´8 C05D 5C¤;DäGC BDÄ DC´0C2 -4ÆV b C D CD"5D BC$CDäICT9 Cä?CT@C FC,,8.D
/// ATAC´8 D2 >C05D 0Dd8C, CCd5 DD8Ct8Dt5D :C, 5D BDÄ 4CäGCT@C08CR 2CTBC¤8 C" At#, C0@CT2D 0D"0D$L CTQ C
/// </summary>ð
public static bool CanChangeLeafStatus(bool hasChildren, bool requestedIsLeaf)ð
{ð
    if (hasChildren && requestedIsLeaf)ð
    {ð
        return false;ð
    }ð
ð
    return true;ð
}ð
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalOperationParameter/
TechnologicalOperationParameterBase.cs">
using Promatis.Net.Domain;ð
using Promatis.Net.Domain.Interface;ð
using Promatis.Net.MES.Domain.Interface;ð
ð
namespace Promatis.Net.MES.Domain;ð
ð
/// <summary>ð
/// B 2Dô7D4ND"0Dò AD4IC0>D BDÄ ´ C0>C48CR0:Cä0<C0>C48CĚ² <CT6CDC D$5DT=Cä;Cä3C,,GCTAC¤8CÄ8 Cä?CT@C FC,,0CÄ8 C,
/// Aä?D 5CD5C´0CTB D$@CT1Cä2C =C,,O C¢ 7C ?Cä;C05C08Dä ?C @C <CTBD >C" 4C´O C¤>C0:D 5D$=Cä9 Cä?CT@C FC,,8.D
/// </summary>ð
public abstract class TechnologicalOperationParameterBase<TOperation, TParameter> : DomainObject,
ISoftDeletableð
    where TOperation : DomainObject, ITechnologicalOperationð
    where TParameter : DomainObject, ITechnologicalParameterð
{ð
    public Guid OperationId { get; set; }ð
ð
    /// <summary>ð
    /// Aô0C$8C40Dd8Cä=C0>CR AC$>C"AD$2Cä : C 0Ct>C$>C´ >Cô5D 0Dd8C,í
    /// </summary>ð
    public virtual TOperation Operation { get; set; } = null!;ð
ð
    public Guid ParameterId { get; set; }ð
ð
    /// <summary>ð
    /// Aô0C$8C40Dd8Cä=C0>CR AC$>C"AD$2Cä : C 0Ct>C$>CÄC C00D 0CÄ5D$@D2í
    /// </summary>ð
    public virtual TParameter Parameter { get; set; } = null!;ð
ð
    /// <summary>ð
    /// Aô@C,,7C00C¢ BCä3CäÂ GD$> C00D 0CÄ5D$@ Cä1Dô7C BCT;CT= CD;Dò 7C ?Cä;C05C08Dòí
    /// </summary>ð
    public bool IsRequired { get; set; } = false;ð

```

```

Ð
    /// <summary>Ð
    /// AÔ>CÄ8CÔ0C´LCÔ>CR 7CÔ0Dt5CÔ8CR ?C @C <CTBD 0 CÔ> D4<Cä;Dt0CÔ8Dâí
    /// </summary>Ð
    public string? DefaultValue { get; set; }Ð
Ð
    /// <summary>Ð
    /// AA8CÔ8CÄ0C´LCÔ> CD>CÔCD BC,,<Cä5 Ct=C GCT=C,,5.Ð
    /// </summary>Ð
    public double? MinValue { get; set; }Ð
Ð
    /// <summary>Ð
    /// AÄ0C¤AC,,<C ;DÄ=Cä 4Cä?D4AD$8CÄ>CR 7CÔ0Dt5CÔ8CRí
    /// </summary>Ð
    public double? MaxValue { get; set; }Ð
Ð
    public DateTime? DeletedAt { get; set; }Ð
    public string? DeletedBy { get; set; }Ð
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalOperationParameter/
TechnologicalOperationParameterBaseValidator.cs">
using FluentValidation;Ð
using Promatis.Net.Domain;Ð
using Promatis.Net.MES.Domain.Interface;Ð
Ð
namespace Promatis.Net.MES.Domain;Ð
Ð
public abstract class TechnologicalOperationParameterBaseValidator<T, TOperation, TParameter> :
DomainObjectValidator<T>Ð
    where T : TechnologicalOperationParameterBase<TOperation, TParameter>Ð
    where TOperation : DomainObject, ITechnologicalOperationÐ
    where TParameter : TechnologicalParameterBaseÐ
{Ð
    protected TechnologicalOperationParameterBaseValidator()Ð
    {Ð
        RuleFor(x => x.OperationId)Ð
            .NotEmpty()Ð
            .WithMessage("A,,4CT=D$8DD8C¤0D$>D >CÔ5D 0Dd8C, >C 0Ct0D$5C´5CÔâ""½
Ð
        RuleFor(x => x.ParameterId)Ð
            .NotEmpty()Ð
            .WithMessage("A,,4CT=D$8DD8C¤0D$>D BCTECÔ>C´>C48Dt5D :Cä3Cä ?C @C <CTBD 0 Cä1Dô7C BCT;CT=.");Ð
Ð
        // A¤@CÄAD Ô?Cä;CT2C 0 C$0C´8CD0Dd8Dó¢ C :D 8CÄ0C´LCÔ>CR 7CÔ0Dt5CÔ8CR =CR <Cä6CTB C KD$L CÄ5CÔLD,,5 C
        RuleFor(x => x.MaxValue)Ð
            .GreaterThanOrEqualTo(x => x.MinValue)Ð
            .When(x => x.MinValue.HasValue && x.MaxValue.HasValue)Ð
            .WithMessage("AÄ0C¤AC,,<C ;DÄ=Cä 4Cä?D4AD$8CÄ>CR 7CÔ0Dt5CÔ8CR =CR <Cä6CTB C KD$L CÄ5CÔLD,,5 CÄ8CÔ8C
    }Ð
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalOperationUnit/TechnologicalOperationUnitBase.cs">
using Promatis.Net.Domain;Ð
using Promatis.Net.Domain.Interface;Ð
using Promatis.Net.MES.Domain.Interface;Ð
Ð
namespace Promatis.Net.MES.Domain;Ð
Ð
/// <summary>Ð
/// A 1D BD 0C¤BCÔ0Dò 1C 7C 4C´O D 2Dô7C, ,,@C AD,,8D OCT<C 0>Ð
/// </summary>Ð
public abstract class TechnologicalOperationUnitBase<TOperation> : DomainObject, ISoftDeletableÐ
    where TOperation : DomainObject, ITechnologicalOperationÐ
{Ð
    public Guid OperationId { get; set; }Ð
Ð
    /// <summary>Ð
    /// AÔ0C$8C40Dd8Cä=CÔ>CR AC$>C"AD$2Cä : Cä?CT@C FC,,8 (C CCD5D" 7C :D KD$> C¤>CÔ:D 5D$=D´< D$8Cô>CÄ 2 MDM)
    /// </summary>Ð
    public virtual TOperation Operation { get; set; } = null!;Ð
Ð
    public Guid UnitId { get; set; }Ð
Ð

```

```

    /// <summary>ð
    /// Að@Dð<C O D AD';Cð0 CÔ0 C 0Ct>C$KC' :C'0D A Cä1Cä@D44Cä2C =C,,O. ð
    /// EF Core C 2D$>CÄ0D$8Dt5D :C, AC$0Cd5D" MD$> D BC 1C'8Dd5C' >C >D CCD>C$0C08Dð ?Cä ACä3C'0D,,5C08Dð<.D
    /// </summary>ð
    public virtual UnitBase Unit { get; set; } = null!;ð
ð
    public int Priority { get; set; } = 1;ð
ð
    public DateTime? DeletedAt { get; set; }ð
    public string? DeletedBy { get; set; }ð
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalOperationUnit/
TechnologicalOperationUnitBaseValidator.cs">
using FluentValidation;ð
using Promatis.Net.Domain;ð
using Promatis.Net.MES.Domain.Interface;ð
ð
namespace Promatis.Net.MES.Domain;ð
ð
    /// <summary>ð
    /// A 0Ct>C$KC' 2C ;C,,4C BCä@ D 2Dô7C•
    /// </summary>ð
    public abstract class TechnologicalOperationUnitBaseValidator<T, TOperation> :
    DomainObjectValidator<T>ð
        where T : TechnologicalOperationUnitBase<TOperation>ð
        where TOperation : DomainObject, ITechnologicalOperationð
    {ð
        protected TechnologicalOperationUnitBaseValidator()ð
        {ð
            RuleFor(x => x.OperationId)ð
                .NotEmpty()ð
                .WithMessage("A,,4CT=D$8DD8Cð0D$>D >C05D 0Dd8C, >C 0Ct0D$5C'5C0â""½
ð
            RuleFor(x => x.UnitId)ð
                .NotEmpty()ð
                .WithMessage("A,,4CT=D$8DD8Cð0D$>D >C >D CCD>C$0C08Dð >C 0Ct0D$5C'5C0â""½
ð
            RuleFor(x => x.Priority)ð
                .InclusiveBetween(1, 10)ð
                .WithMessage("Að@C,,>D 8D$5D" >C >D CCD>C$0C08Dð 4Cä;Cd5C0 1D'BDÂ 2 CD8C ?C 7Cä=CR >D" 4Cä ä""
        }ð
    }
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalParameter/TechnologicalParameterBase.cs">
using Promatis.Net.Domain;ð
using Promatis.Net.MES.Domain.Interface;ð
ð
namespace Promatis.Net.MES.Domain;ð
ð
    /// <summary>ð
    /// A 0Ct>C$KC' BCTEC0>C'>C48Dt5D :C,,9 C00D 0CÄ5D$@ (D ?D 0C$>Dt=C,,: DT0D 0CðBCT@C,,AD$8Cð ?D >Dd5D ACä2).ð
    /// </summary>ð
    public abstract class TechnologicalParameterBase : ReferenceBase, ITechnologicalParameterð
    {ð
        /// <summary>ð
        /// A$=CTHC08C' :C'NDr =C ACô@C 2CäGC08Cð 5CD8C08Db 8Ct<CT@CT=C,,O.ð
        /// </summary>ð
        public Guid? UnitOfMeasurementId { get; set; }ð
ð
        public virtual UnitOfMeasurement? UnitOfMeasurement { get; set; }ð
ð
        /// <summary>ð
        /// B$8C0 4C =C0KDR ?C @C <CTBD 0 (C00C0@C,,<CT@: Bt8D ;CäÄ !D$@Cä:C Ä D4;CT2Cä'1
        /// </summary>ð
        public string DataType { get; set; } = "Numeric";ð
ð
        /// <summary>ð
        /// B 0Ct@CTHCT=C0KCR <CTBCä4D² @C AD GE BC
        /// </summary>ð
        public CalculationMethod AllowedMethods { get; set; } = CalculationMethod.None;ð
    }
</file>

```

```

<file path="Promatis.Net.MES.Domain/TechnologicalParameter/TechnologicalParameterBaseValidator.cs">
using FluentValidation;
using Promatis.Net.Domain;

namespace Promatis.Net.MES.Domain;

public abstract class TechnologicalParameterBaseValidator<T> : ReferenceBaseValidator<T>
    where T : TechnologicalParameterBase
{
    private static readonly string[] AllowedDataTypes = ["Numeric", "String", "Boolean",
"DateTime"];

    protected TechnologicalParameterBaseValidator() : base()
    {
        // 1. A$0C'8CD0Dd8D0 5CD8C08DdK C,,7CÄ5D 5C08D0 „4CäAD$CC0=C Â BC : C0C0 B r MD$> TechnologicalParame
        RuleFor(x => x.UnitOfMeasurement)
            .NotNull()
            .WithMessage("AT4C,,=C,,FC 8Ct<CT@CT=C,,O C05 CÄ>Cd5D" 1D'BDÂ çVÆÂ""½

        // 2. A$0C'8CD0Dd8D0 BC,,?C 4C =C0KD]
        RuleFor(x => x.DataType)
            .NotEmpty()
            .WithMessage("B$8C0 4C =C0KDR ?C @C <CTBD 0 Cä1D07C BCT;CT=.")
            .Must(type => AllowedDataTypes.Contains(type))
            .WithMessage($"A05CD>C0CD BC,,<D'9 D$8C0 4C =C0KDRâ C 7D 5D,,5C0=D'5 Ct=C GCT=C,,0: {string.Join(",
    }
}
</file>

```

```

<file path="Promatis.Net.MES.Domain/TechnologicalParameterCalcMethod/
TechnologicalParameterCalcMethodBase.cs">
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Promatis.Net.MES.Domain.Interface;

namespace Promatis.Net.MES.Domain;

public abstract class TechnologicalParameterCalcMethodBase<TUnit, TOperation, TParameter> :
DomainObject, ISoftDeletable
    where TUnit : UnitBase
    where TOperation : DomainObject, ITechnologicalOperation
    where TParameter : TechnologicalParameterBase
{
    public Guid UnitId { get; set; }

    /// <summary>
    /// Bd5DT>C$0D0 5CD8C08Dd0
    /// </summary>
    public virtual TUnit Unit { get; set; } = null!;

    public Guid TechnologicalOperationId { get; set; }

    /// <summary>
    /// B$5DT=Cä;Cä3C,,GCTAC0D0 >C05D 0Dd8Dý
    /// </summary>
    public virtual TOperation TechnologicalOperation { get; set; } = null!;

    public Guid TechnologicalParameterId { get; set; }

    /// <summary>
    /// B$5DT=Cä;Cä3C,,GCTAC08C' ?C @C <CTBD
    /// </summary>
    public virtual TParameter TechnologicalParameter { get; set; } = null!;

    /// <summary>
    /// AÄ5D$>CB @C ADt5D$0
    /// </summary>
    public CalculationMethod CalculationMethod { get; set; } = CalculationMethod.None;

    public DateTime? DeletedAt { get; set; }
    public string? DeletedBy { get; set; }
}
</file>

```

```

<file path="Promatis.Net.MES.Domain/TechnologicalParameterCalcMethod/
TechnologicalParameterCalcMethodBaseValidator.cs">

```

```

using FluentValidation;
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain.Interface;

namespace Promatis.Net.MES.Domain;

public abstract class TechnologicalParameterCalcMethodBaseValidator<T, TUnit, TOperation,
TParameter> : DomainObjectValidator<T>
    where T : TechnologicalParameterCalcMethodBase<TUnit, TOperation, TParameter>
    where TUnit : UnitBase
    where TOperation : DomainObject, ITechnologicalOperation
    where TParameter : TechnologicalParameterBase
{
    protected TechnologicalParameterCalcMethodBaseValidator()
    {
        RuleFor(x => x.UnitId)
            .NotEmpty().WithMessage("A,,4CT=D$8DD8C=0D$>D FCTECä2Cä9 CT4C,,=C,,FD² >C 0Ct0D$5C´5C0â""½
        RuleFor(x => x.TechnologicalOperationId)
            .NotEmpty().WithMessage("A,,4CT=D$8DD8C=0D$>D BCTECÔ>C´>C48Dt5D :Cä9 Cä?CT@C FC,,8 Cä1D07C BCT;CT=
        RuleFor(x => x.TechnologicalParameterId)
            .NotEmpty().WithMessage("A,,4CT=D$8DD8C=0D$>D BCTECÔ>C´>C48Dt5D :Cä3Cä ?C @C <CTBD 0 Cä1D07C BCT;
        // A0@Cä2CT@D05CÄÄ GD$> CÄ5D$>CB @C Adt5D$0 C KC² OC$=Cä 2D´1D 0C0 ?Cä;DÄ7Cä2C BCT;CT<D
        RuleFor(x => x.CalculationMethod)
            .NotEqual(CalculationMethod.None)
            .WithMessage("A05Cä1DT>CD8CÄ> C$KC @C BDÄ :Cä@D 5C=BC0KC´ <CTBCä4 D 0D GCTBC ?C @C <CTBD >C"ä""½
    }
}
</file>

```

```

<file path="Promatis.Net.MES.Domain/TechnologicalParameterValue/TechnologicalParameterValueBase.cs">
using Promatis.Net.Domain;

namespace Promatis.Net.MES.Domain;

public abstract class TechnologicalParameterValueBase<TUnit, TParameter> : DomainObject
    where TUnit : UnitBase
    where TParameter : TechnologicalParameterBase
{
    public Guid UnitId { get; set; }
    /// <summary>
    /// Bd5DT>C$0D0 5CD8C08Dd0 (C,,AD$>Dt=C,,: CD0C0=D´E)
    /// </summary>
    public virtual TUnit Unit { get; set; } = null!;
    public Guid TechnologicalParameterId { get; set; }
    /// <summary>
    /// B$5DT=Cä;Cä3C,,GCTAC=8C´ ?C @C <CTBD
    /// </summary>
    public virtual TParameter TechnologicalParameter { get; set; } = null!;
    /// <summary>
    /// B KD >CR ,3D 0Ct=Cä5") Ct=C GCT=C,,5, C05D 5CD0C0=Cä5 CäB SCADA/IoT C,,;C, >C05D 0D$>D 0D
    /// </summary>
    public string Value { get; set; } = string.Empty;
    /// <summary>
    /// B$>Dt=Cä5 C$@CT<D0 DC,,:D 0Dd8C, ?C @C <CTBD 0 (Time-Series CÄ5D$:C •
    /// </summary>
    public DateTime Date { get; set; } = DateTime.UtcNow;
}
</file>

```

```

<file path="Promatis.Net.MES.Domain/TechnologicalParameterValue/
TechnologicalParameterValueBaseValidator.cs">
using FluentValidation;
using Promatis.Net.Domain;

namespace Promatis.Net.MES.Domain;

public abstract class TechnologicalParameterValueBaseValidator<T, TUnit, TParameter> :
DomainObjectValidator<T>

```

```

where T : TechnologicalParameterValueBase<TUnit, TParameter>
where TUnit : UnitBase
where TParameter : TechnologicalParameterBase
{
    protected TechnologicalParameterValueBaseValidator()
    {
        RuleFor(x => x.UnitId)
            .NotEmpty().WithMessage("A,,4CT=D$8DD8C=0D$>D FCTECä2Cä9 CT4C,,=C,,FD² >C 0Ct0D$5C´5C0â""½

        RuleFor(x => x.TechnologicalParameterId)
            .NotEmpty().WithMessage("A,,4CT=D$8DD8C=0D$>D BCTECÔ>C´>C48Dt5D :Cä3Cä ?C @C <CTBD 0 Cä1D07C BCT;

        RuleFor(x => x.Value)
            .NotNull().WithMessage("At=C GCT=C,,5 C00D 0CÄ5D$@C =CR <Cä6CTB C KD$L null.")
            // B 0Ct@CTHC 5CÄ ?D4AD$CDâ AD$@Cä:D2Â BC : C=0C¢ 4C´O String-C00D 0CÄ5D$@Cä2 D0BCâ <Cä6CTB C KD$
            // C0> D41C,,@C 5CÄ AC´CDt0C"=D´5 C0@Cä1CT;D² ?Cä :D 0D0<D
            .Must(val => val == null || val.Trim() == val)
            .WithMessage("At=C GCT=C,,5 C05 CÄ>Cd5D" =C GC,,=C BDÄAD0 8C´8 Ct0C=0C0GC,,2C BDÄAD0 ?D >C 5C´0CÄ8."

        RuleFor(x => x.Date)
            .Must(date => date <= DateTime.UtcNow.AddSeconds(5))
            .WithMessage("A$@CT<D0 DC,,:D 0Dd8C, ?C @C <CTBD 0 C05 CÄ>Cd5D" 1D´BDÄ 2 C CCDCD"5CÄâ""½
    }
}
</file>

<file path="Promatis.Net.MES.Domain/UnitBase/UnitBase.cs">
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Promatis.Net.MES.Domain.Interface;
{
    namespace Promatis.Net.MES.Domain;
    {
        /// <summary>
        /// A 1D BD 0C=BC00D0 1C 7C 4C´O C$ACTE Cä@C40C08Ct0Dd8Cä=C0KDR 8 C0@Cä8Ct2Cä4D BC$5C0=D´E CT4C,,=C,,F.D
        /// </summary>
        public abstract class UnitBase : ReferenceTreeBase<UnitBase>, IAudit
        {
            public UnitKind Kind { get; init; }
            public required UnitType Type { get; set; }
        }
    }
</file>

<file path="Promatis.Net.MES.Domain/UnitBase/UnitBaseValidator.cs">
using FluentValidation;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
{
    namespace Promatis.Net.MES.Domain;
    {
        /// <summary>
        /// A 1D BD 0C=BC0KC´ 2C ;C,,4C BCä@ CD;D0 1C 7Cä2Cä3Cä :C´0D AC Væ-D& 6R1
        /// </summary>
        /// <typeparam name="T">B$8C0Ä =C AC´5CDCCT<D´9 CäB UnitBase.</typeparam>
        public abstract class UnitBaseValidator<T> : ReferenceBaseValidator<T>
        where T : UnitBase
        {
            protected UnitBaseValidator() : base()
            {
                // A0@Cä2CT@C=0, DtBCä 2 Type C05 C0@C,,HC´> C0CD BCä5/CD5DD>C´BC0>CR 7C00Dt5C08CR DC´0C4>C-
                RuleFor(x => x.Type).NotEmpty();

                // A0@Cä2CT@C=0 D >CäBC$5D$AD$2C,,0 A=0D$5C4>D 8C, ,,¶-æB´ 8 B$8C00 (Type) C00 CäAC0>C$5 C 8D$>C$KDR <C
                RuleFor(x => x)
                    .Must(u => ((int)u.Kind & (int)u.Type) != 0)
                    .WithMessage("B$8C0 =CR ACä>D$2CTBD BC$CCTB C=0D$5C4>D 8C,â""½
            }
        }
    }
</file>

<file path="Promatis.Net.MES.Domain/UnitBase/UnitHierarchyEngine.cs">
using Promatis.Net.MES.Domain.Interface;
{
    namespace Promatis.Net.MES.Domain;
    {
        public static class UnitHierarchyEngine
    }
}

```

```

{
    /// <summary>
    /// A4;Cä1C ;DÄ=D´9 C,,AD$>Dt=C,,: Cö@C 2CDK: Cö@Cä2CT@Dö5D"Â 2C ;C,,4Cö> C´8 C$;Cä6CT=C,,5 D 5C 5Cö:C 2 D >
    /// A,,ACö>C´LctCCTBD O Cö0CÄ5D BC$> C=0C¢ 2 B #A -D$@C,,3C45D 0DRÂ BC : C, 2 UI-C$0C´8CD0Dd8C,í
    /// </summary>
    public static bool IsHierarchyValid(UnitKind parentKind, UnitKind childKind)
    {
        // 1. B$5D <C,,=C ;DÄ=D´9 D47CT; Position Cö8C= >C44C =CR <Cä6CTB C KD$L D >CD8D$5C´5Cí
        if (parentKind == UnitKind.Position) return false;

        // 2. AD5Cö0D BC <CT=D" <Cä6CTB D >CD5D 6C BDÂ 2D Q, C= @Cä<CR ÷6-F-öí
        if (parentKind == UnitKind.Department) return childKind != UnitKind.Position;

        // 3. B ?CTFC,,0C´8Ct8D >C$0Cö=D´5 Ct>CöK C,,7Cä;C,,@Cä2C =D² 4D CC2 >D" 4D CC40, Cö> CÄ>C4CD" ACä4CT@Cd
        if (parentKind is UnitKind.Production or UnitKind.Transport or UnitKind.Storage)
        {
            return childKind == parentKind || childKind == UnitKind.Position;
        }

        return true;
    }
}

/// <summary>
/// A$>Ct2D 0D"0CTB D ?C,,ACä: Aä Aä B Aö B´% D$8Cö>C" ...Væ-EG- R´ 4C´O C$KCö0CD0DäICT3Cä ACö8D :C 2 UI,D
/// CäACö>C$KC$0DöADÄ =C C BCT3Cä@C,,8 (UnitKind) D >CD8D$5C´LD :Cä3Cä CCT;C Ä 2 C= >D$>D KC´ <D² ACT9Dt0
/// </summary>
public static IEnumerable<UnitType> GetAllowedChildTypes(UnitKind parentKind)
{
    var allTypes = (UnitType[])Enum.GetValues(typeof(UnitType));
    var allowedTypes = new List<UnitType>();

    foreach (UnitType type in allTypes)
    {
        if (type == UnitType.None) continue;

        // 1. B =C GC ;C =C ECä4C,,<, C¢ :C :Cä<D2 ?CäBCT=Dd8C ;DÄ=Cä<D2 ¶-æB >D$=CäAC,,BD O DöBCäB D$8Cöí
        // B :C =C,,@D45CÄ 2D 5 C=0D$5C4>D 8C,Ä GD$>C K Cö>Cö0D$L, C" :C :D4N CÄ0D :D2 ?Cä?C 4C 5D" MD$>D"
        foreach (UnitKind potentialChildKind in (UnitKind[])Enum.GetValues(typeof(UnitKind)))
        {
            // ATAC´8 D$8Cö ?Cä1C,,BCä2Cä 2DT>CD8D" 2 DöBD2 :C BCT3Cä@C,,Nö
            if (((int)potentialChildKind & (int)type) != 0)
            {
                // A, 5D ;C, ?D 0C$8C´0 C,,5D 0D EC,,8 B At AT(A .B" 2C´>Cd8D$L DöBD2 :C BCT3Cä@C,,N C" BCT
                if (IsHierarchyValid(parentKind, potentialChildKind))
                {
                    if (!allowedTypes.Contains(type))
                    {
                        allowedTypes.Add(type);
                    }
                }
            }
        }
    }

    return allowedTypes;
}
}
</file>

```

```

<file path="Promatis.Net.MES.Domain/UnitOfMeasurement/UnitOfMeasurement.cs">
using Promatis.Net.Domain;
{
    namespace Promatis.Net.MES.Domain;
    {
        /// <summary>
        /// AD>CÄ5Cö=C O D CD"=CäAD$L CT4C,,=C,,FD² 8Ct<CT@CT=C,,O (Aö!A,' 4C´O D$5DT=Cä;Cä3C,,GCTAC=8DR ?C @C <CTBD >C"
        /// </summary>
        public class UnitOfMeasurement : ReferenceBase
        {
            // B 2Cä9D BC$0 Id, Code, Name, Description Cö0D$8C$=Cä CCö0D ;CT4Cä2C =D² >D" &VfW&Væ6T& 6Rí
            // Code - C= @C BC= >CR >C >Ct=C GCT=C,,5 (Cö0Cö@C,,<CT@, "°C", "CÄ<", "Cä1/CÄ8Cö"´í
            // Name - Cö>C´=Cä5 Cö0C,,<CT=Cä2C =C,,5 (Cö0Cö@C,,<CT@, "A4@C 4D4A Bd5C´LD 8Dò"Â $ C,,;C´8CÄ5D$@").
        }
    }
</file>

```

```

<file path="Promatis.Net.MES.Domain/UnitOfMeasurement/UnitOfMeasurementValidator.cs">

```

```

using FluentValidation;
using Promatis.Net.Domain;

namespace Promatis.Net.MES.Domain;

public class UnitOfMeasurementValidator : ReferenceBaseValidator<UnitOfMeasurement>
{
    // A05D 5Cä?D 5CD5C´OCT< C 0Ct>C$CDâ @CT3D4;Dô@C=C CÔ0 D 0D HC,,@CT=CÔCDâ ,,@D4A/C´0D"Â FC,,DD K, Cô@Cä1CT;D
    protected override string CodeRegexPattern => @"^[\p{L}\p{N}_\.\°\/\*\-\s\%]*$";
    
    protected override string CodeValidationMessage => "A@C BC@>CR >C >Ct=C GCT=C,,5 CÄ>Cd5D" ACä4CT@Cd0D$L C

    public UnitOfMeasurementValidator() : base()
    {
        // Aô@C 2C,,;Cä æ÷DV× G´ 8 Matches D46CR >D$@C 1CäBC ND" 2 C 0Ct>C$>CÂ :C´0D ACR ?Câ =C HCT<D2 =Cä2Cä<

        RuleFor(x => x.Code)
            .NotEmpty().WithMessage("A@C BC@>CR >C >Ct=C GCT=C,,5 Cä1Dô7C BCT;DÄ=Cä 4C´O Ct0Cô>C´=CT=C,,0.")
            .MaxLength(15).WithMessage("A@C BC@>CR >C >Ct=C GCT=C,,5 CÔ5 CD>C´6CÔ> Cô@CT2D´HC BDÄ R AC,,<

        RuleFor(x => x.Name)
            .MaxLength(100).WithMessage("Aô>C´=Cä5 CÔ0C,,<CT=Cä2C =C,,5 CT4C,,=C,,FD² 8Ct<CT@CT=C,,0 CÔ5 CD>C´

    }
}
</file>

<file path="Promatis.Net.MES.Service/Promatis.Net.MES.Service.csproj">
<Project Sdk="Microsoft.NET.Sdk">
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
    </PropertyGroup>
    <ItemGroup>
        <ProjectReference Include="..\..\Core\Service\Promatis.Net.Service.csproj" />
        <ProjectReference Include="..\Promatis.Net.MES.Data\Promatis.Net.MES.Data.csproj" />
    </ItemGroup>
</Project>
</file>

<file path="Promatis.Net.MES.Service/TechnologicalOperationParameterService/
ITechnologicalOperationParameterService.cs">
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;
using Promatis.Net.Service;

namespace Promatis.Net.MES.Service;

public interface ITechnologicalOperationParameterService<T, TOperation, TParameter> :
IBaseService<T, Guid>
    where T : TechnologicalOperationParameterBase<TOperation, TParameter>
    where TOperation : DomainObject, ITechnologicalOperation
    where TParameter : TechnologicalParameterBase
{
    /// <summary>
    /// Aô>C´CDt8D$L C$ACR ?C @C <CTBD K, Cô@C,,2Dô7C =CÔKCR : C@>CÔ:D 5D$=Cä9 Cä?CT@C FC,,8
    /// </summary>
    Task<List<T>> GetByOperationIdAsync(Guid operationId);

    /// <summary>
    /// Aô>C´CDt8D$L D$>C´LC@> Cä1Dô7C BCT;DÄ=D´5 Cô0D 0CÄ5D$@D² 4C´O Cä?CT@C FC,,8
    /// </summary>
    Task<List<T>> GetRequiredParametersByOperationIdAsync(Guid operationId);
}
</file>

<file path="Promatis.Net.MES.Service/TechnologicalOperationParameterService/
TechnologicalOperationParameterService.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Internal;
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain;

```



```

using Promatis.Net.MES.Domain.Interface;
using Promatis.Net.Service;
namespace Promatis.Net.MES.Service;
public abstract class TechnologicalOperationParameterService<T, TOperation, TParameter, TContext>(
    IDbContextFactory<TContext> contextFactory)
    : BaseService<T, Guid, TContext>(contextFactory), ITechnologicalOperationParameterService<T,
TOperation, TParameter>
    where T : TechnologicalOperationParameterBase<TOperation, TParameter>
    where TOperation : DomainObject, ITechnologicalOperation
    where TParameter : TechnologicalParameterBase
    where TContext : DbContext
{
    public async Task<List<T>> GetByOperationIdAsync(Guid operationId)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();

        return await context.Set<T>()
            .Where(x => x.OperationId == operationId)
            .Include(x => x.Parameter) // B @C 7D2 ?Cä4D$0C48C$0CT< D ?D 0C$>Dt=C,,; Cö0D 0CÄ5D$@Cä2 CD;Dò T•
            .AsNoTracking()
            .ToListAsync();
    }

    public async Task<List<T>> GetRequiredParametersByOperationIdAsync(Guid operationId)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();

        return await context.Set<T>()
            .Where(x => x.OperationId == operationId && x.IsRequired)
            .Include(x => x.Parameter)
            .AsNoTracking()
            .ToListAsync();
    }
}
</file>

```

```

<file path="Promatis.Net.MES.Service/TechnologicalOperationService/
ITechnologicalOperationService.cs">
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;
using Promatis.Net.Service;
namespace Promatis.Net.MES.Service;
public interface ITechnologicalOperationService<T, TLink> : ITreeService<T>
    where T : ReferenceTreeBase<T>, ITechnologicalOperation,
Promatis.Net.Domain.Interface.IDomainObjectHasKey<Guid>
    where TLink : class
{
    /// <summary>
    /// Aö>C´CDt8D$L D ?C,,ACä: D 0Ct@CTHCT=CÖ>C4> Cä1Cä@D44Cä2C =C,,0 (Dä=C,,BCä2) CD;Dò :Cä=Cq@CTBCÖ>C' BCTECÖ
    /// </summary>
    Task<List<UnitBase>> GetAllowedUnitsAsync(Guid operationId);
}
</file>

```

```

<file path="Promatis.Net.MES.Service/TechnologicalOperationService/
TechnologicalOperationService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;
using Promatis.Net.Service;
namespace Promatis.Net.MES.Service;
public abstract class TechnologicalOperationService<T, TLink, TContext>(IDbContextFactory<TContext>
contextFactory)
    : ReferenceTreeService<T, TContext>(contextFactory), ITechnologicalOperationService<T, TLink>
    where T : ReferenceTreeBase<T>, ITechnologicalOperation, IDomainObjectHasKey<Guid>, new()
    where TLink : TechnologicalOperationUnitBase<T>
    where TContext : DbContext

```

```

{
    public async Task<List<UnitBase>> GetAllowedUnitsAsync(Guid operationId)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();

        // LINQ-Ct0C0@CâA CâBD 0C 0D$K$0CTB C,,4CT0C`LC0>:D
        // A ;C 3Câ4C @D0 BCâ<D2Â GD$> C" FV6†æöÆöv-6 Ä÷ W& F-öâVæ-D& 6R AC$>C"AD$2Câ Væ-B
        // C,,<CT5D" AD$@Câ3C,,9 D$8C0 Væ-D& 6RÂ Tb 6÷&R ?CâAD$@Câ8D" GC,,AD$KC' 5 Â0÷ô"â 0C$BCâ<C BC,,GCTAC÷8.D
        return await context.Set<TLink>()
            .Where(x => x.OperationId == operationId)
            .Select(x => x.Unit)
            .AsNoTracking()
            .ToListAsync();
    }
}

/// <summary>
/// AâAD$0C$;D05CÂ <CTBCâ4 C 1D BD 0C÷BC0KCÂ =C BCT:D4ICT< D4@Câ2C05. A÷>C0:D 5D$=D'9 C0@C,,:C'0CD=Câ9 D
/// D$5DT?D >Dd5D ACâ2 D 0CÂ ?CT@CT>C0@CT4CT;C,,B CT3Câ 4C'0 D >Ct4C =C,,0 D 2Câ8DR ?Câ;C,,<Câ@DD=D'E Câ?CT@
/// </summary>
public abstract override Task<T> CreateChildTemplateAsync(T parent);
}
</file>

<file path="Promatis.Net.MES.Service/TechnologicalParameterCalcMethodService/
ITechnologicalParameterCalcMethodService.cs">
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;
using Promatis.Net.Service;

namespace Promatis.Net.MES.Service;

public interface ITechnologicalParameterCalcMethodService<T, TUnit, TOperation, TParameter> :
IBaseService<T, Guid>
    where T : TechnologicalParameterCalcMethodBase<TUnit, TOperation, TParameter>
    where TUnit : UnitBase
    where TOperation : DomainObject, ITechnologicalOperation
    where TParameter : TechnologicalParameterBase
{
    /// <summary>
    /// A00C"BC, :Câ=C÷@CTBC0CDâ 8C0AD$@D4:Dd8Dâ @C ADt5D$0 CD;D0 BD >C":Cf C >D CCD>C$0C08CR ² C05D 0Dd8D
    /// </summary>
    Task<T?> GetCalcMethodAsync(Guid unitId, Guid operationId, Guid parameterId);

    /// <summary>
    /// A0>C`CDt8D$L C$ACR <CTBCâ4D² @C ADt5D$0 CD;D0 :Câ=C÷@CTBC0>C' >C05D 0Dd8C, =C >C >D CCD>C$0C08C•
    /// </summary>
    Task<List<T>> GetMethodsByOperationAndUnitAsync(Guid operationId, Guid unitId);
}
</file>

<file path="Promatis.Net.MES.Service/TechnologicalParameterCalcMethodService/
TechnologicalParameterCalcMethodService.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Internal;
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;
using Promatis.Net.Service;

namespace Promatis.Net.MES.Service;

public abstract class TechnologicalParameterCalcMethodService<T, TUnit, TOperation, TParameter,
TContext>()
    where T : TechnologicalParameterCalcMethodBase<TUnit, TOperation, TParameter>
    where TUnit : UnitBase
    where TOperation : DomainObject, ITechnologicalOperation
    where TParameter : TechnologicalParameterBase
    where TContext : DbContext
{
    public async Task<T?> GetCalcMethodAsync(Guid unitId, Guid operationId, Guid parameterId)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
    }
}

```

```

    return await context.Set<T>()
        .FirstOrDefaultAsync(x => x.UnitId == unitId &&
            x.TechnologicalOperationId == operationId &&
            x.TechnologicalParameterId == parameterId);
    }
}

public async Task<List<T>> GetMethodsByOperationAndUnitAsync(Guid operationId, Guid unitId)
{
    await using TContext context = await ContextFactory.CreateDbContextAsync();

    return await context.Set<T>()
        .Where(x => x.TechnologicalOperationId == operationId && x.UnitId == unitId)
        .Include(x => x.TechnologicalParameter) // A0>CD3D CCd0CT< D 2D07C =C0KC' ?C @C <CTBD
        .AsNoTracking()
        .ToListAsync();
}
}
</file>

<file path="Promatis.Net.MES.Service/TechnologicalParameterService/
ITechnologicalParameterService.cs">
using Promatis.Net.MES.Domain;
using Promatis.Net.Service;
namespace Promatis.Net.MES.Service;
public interface ITechnologicalParameterService<T> : IReferenceService<T>
    where T : TechnologicalParameterBase
{
    /// <summary>
    /// A0>C'CDt8D$L C00D 0CÄ5D$@D²Â >D$DC,,;DÄBD >C$0C0=D'5 C0> D$8C0C CD0C0=D'E (Numeric, String, Boolean, D
    /// </summary>
    Task<List<T>> GetByDataTypeAsync(string dataType);
}
</file>

<file path="Promatis.Net.MES.Service/TechnologicalParameterService/
TechnologicalParameterService.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Internal;
using Promatis.Net.MES.Domain;
using Promatis.Net.Service;
namespace Promatis.Net.MES.Service;
public abstract class TechnologicalParameterService<T, TContext>(IDbContextFactory<TContext>
contextFactory)
    : ReferenceService<T, TContext>(contextFactory), ITechnologicalParameterService<T>
    where T : TechnologicalParameterBase
    where TContext : DbContext
{
    public async Task<List<T>> GetByDataTypeAsync(string dataType)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();

        return await context.Set<T>()
            .Where(x => x.DataType == dataType)
            .AsNoTracking()
            .ToListAsync();
    }
}
</file>

<file path="Promatis.Net.MES.Service/UnitOfMeasurementService/IUnitOfMeasurementService.cs">
using Promatis.Net.MES.Domain;
using Promatis.Net.Service;
namespace Promatis.Net.MES.Service;
/// <summary>
/// A0>C0BD 0C0B D 5D 2C,,AC0>C4> D ;Cä0 CD;D0 CC0@C 2C'5C08D0 AC0@C 2CäGC08C0>CÄ 5CD8C08Db 8Ct<CT@CT=C,,0.D
/// </summary>
public interface IUnitOfMeasurementService : IReferenceService<UnitOfMeasurement>
{
}

```

</file>

```
<file path="Promatis.Net.MES.Service/UnitOfMeasurementService/UnitOfMeasurementService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.MES.Data;
using Promatis.Net.MES.Domain;
using Promatis.Net.Service;

namespace Promatis.Net.MES.Service;

/// <summary>
/// B 5D 2C,,A D4?D 0C$;CT=C,,O CÔ>D <C BC,,2CÔ>-D ?D 0C$>Dt=Cä9 C,,=DD>D <C FC,,5C' 5CD8C08Db 8Ct<CT@CT=C,,O.D
/// </summary>
public class UnitOfMeasurementService(IDbContextFactory<MesApplicationDbContext> contextFactory)
    : ReferenceService<UnitOfMeasurement, MesApplicationDbContext>(contextFactory),
    IUnitOfMeasurementService
{
}
</file>
```

```
<file path="Promatis.Net.MES.Service/UnitService/IUnitBaseService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;
using Promatis.Net.Service;

namespace Promatis.Net.MES.Service;

public interface IUnitBaseService<TContext> : IReferenceTreeService<UnitBase, TContext> // <- A,,!Aô A A' Aô
D$8C00 C" @Cä4C,,BCT;DÄAC=8C' 8C0BCT@DD5C"A!D
    where TContext : DbContext
{
    Task<List<UnitBase>> GetByKindAsync(UnitKind kind);
    Task<List<UnitBase>> GetByTypeAsync(UnitType type);
    Task<List<UnitBase>> GetByKindAndTypeAsync(UnitKind kind, UnitType type);
}
</file>
```

```
<file path="Promatis.Net.MES.Service/UnitService/UnitBaseService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;
using Promatis.Net.Service;

namespace Promatis.Net.MES.Service;

public abstract class UnitBaseService<TContext>(IDbContextFactory<TContext> contextFactory)
    : ReferenceTreeService<UnitBase, TContext>(contextFactory), IUnitBaseService<TContext>
    where TContext : DbContext
{
    // =====
    // Aô A "BD B AT Aô+A' A !B$ A B$ B' BT#Aç ,, AT A' At#AT"B / Aô B Aä Aô Ad )D
    // =====

    /// <summary>
    /// A= CÔBD 0C=B DD0C @C,,:C,â !C <C @CT0C'8Ct0Dd8D0 1D44CTB C00C08D 0C00 C00 D ;Cä9 C08Cd5 D
    /// C" :Cä=C=CTBC0>CÄ ACT@C$8D 5, C= >D$>D KC' 2C,,4C,,B C0>C'8CÄ>D DC0KCR :C'0D AD² ,,FW 'FÖVçEVæ-B 8 CD@.
    /// </summary>
    public abstract override Task<UnitBase> CreateChildTemplateAsync(UnitBase parent);

    // =====
    // Bt B "B' AÄ B$ AD+ A$+A B A, A Aô+BR ,,!C$>C"AD$2C ¶-æB 8 Type D$5C05D L C$8CD=D² 8CD5C ;DÄ=Cä•
    // =====

    public async Task<List<UnitBase>> GetByKindAsync(UnitKind kind)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        return await context.Set<UnitBase>()
            .AsNoTracking()
            .Where(x => x.Kind == kind)
            .ToListAsync();
    }
}
```

```

    }
    public async Task<List<UnitBase>> GetByTypeAsync(UnitType type)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        // A,,ACô>C`LCtCCT< Cô>C 8D$>C$>CR $ ", D$0C¢ :C : UnitType - DôBCä ´fÆ w5Ÿ
        return await context.Set<UnitBase>()
            .AsNoTracking()
            .Where(x => (x.Type & type) != 0)
            .ToListAsync();
    }
}
public async Task<List<UnitBase>> GetByKindAndTypeAsync(UnitKind kind, UnitType type)
{
    await using TContext context = await ContextFactory.CreateDbContextAsync();
    return await context.Set<UnitBase>()
        .AsNoTracking()
        .Where(x => x.Kind == kind && (x.Type & type) != 0)
        .ToListAsync();
}
}
</file>

<file path="Promatis.Net.MES.Tests/AssemblyInfo.cs">
// AäBC¢;DäGC 5D" ?C @C ;C`5C`LCô>CR 2D´?Cä;Cô5Cô8CR BCTAD$>C" 2 DôBCä9 D 1Cä@C¢5D
[assembly: CollectionBehavior(DisableTestParallelization = true)]
</file>

<file path="Promatis.Net.MES.Tests/Promatis.Net.MES.Tests.csproj">
<Project Sdk="Microsoft.NET.Sdk">
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
        <IsPackable>false</IsPackable>
    </PropertyGroup>
    <ItemGroup>
        <PackageReference Include="coverlet.collector" Version="10.0.0">
            <PrivateAssets>all</PrivateAssets>
            <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</
IncludeAssets>
        </PackageReference>
        <PackageReference Include="FluentAssertions" Version="8.10.0" />
        <PackageReference Include="FluentValidation" Version="12.1.1" />
        <PackageReference Include="Microsoft.EntityFrameworkCore.InMemory" Version="10.0.8" />
        <PackageReference Include="Microsoft.Extensions.Configuration" Version="10.0.8" />
        <PackageReference Include="Microsoft.NET.Test.Sdk" Version="18.5.1" />
        <PackageReference Include="Moq" Version="4.20.72" />
        <PackageReference Include="xunit.runner.visualstudio" Version="3.1.5">
            <PrivateAssets>all</PrivateAssets>
            <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</
IncludeAssets>
        </PackageReference>
        <PackageReference Include="xunit.v3" Version="3.2.2" />
    </ItemGroup>
    <ItemGroup>
        <ProjectReference Include="..\Promatis.Net.Mes.Service\Promatis.Net.MES.Service.csproj" />
    </ItemGroup>
    <ItemGroup>
        <Using Include="Xunit" />
    </ItemGroup>
</Project>
</file>

<file path="Promatis.Net.MES.Tests/Service/UnitServiceStandaloneTests.cs">
using FluentAssertions;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.ChangeTracking;
using Microsoft.Extensions.Configuration;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.MES.Domain.Interface;

```

```

using Promatis.Net.MES.Service;
namespace Promatis.Net.MES.Tests;

public class UnitServiceStandaloneTests
{
    // 1. A,,ACô>C`LCtCCT< C$5Ct4CR FW7DF$6öçFW‡M
    private readonly IDbContextFactory<TestDbContext> _factory;
    private readonly DbContextOptions<TestDbContext> _options;

    public class TestUnit : Domain.UnitBase { }

    /// <summary>
    /// B$5D BCâ2D`9 D 5D 2C,,A CD;Dò 2CT@C,,DC,,:C FC,,8 C 8Ct=CTA-C$KC >D >C$ Væ-D& 6R 2 CÄ>CDCC`LCÔKDR BCTAD$0
    /// A,,!Aô A A` Aô : B 8C4=C BD4@C ?D 8C$5CD5C00 C$ AD$0C04C @D$C UnitBaseService D >CD=C,,< generic-C00
    /// </summary>
    public class TestUnitService(IDbContextFactory<TestDbContext> factory)
        : UnitBaseService<TestDbContext>(factory)
    {
        /// <summary>
        /// Aô@CâAD$5C"HC O Ct0C4;D4HC=0 DD0C @C,,:C, ?Cä;C,,<Câ@DD8Ct<C 4C`O D4ACô5D,,=Câ9 C= >CÄ?C,,;DôFC,,8 C 1
        /// </summary>
        public override Task<Domain.UnitBase> CreateChildTemplateAsync(Domain.UnitBase parent)
        {
            // A,,!Aô A A` Aô : A 5Ct>C00D =Câ5 CD8C00CÄ8Dt5D :Câ5 D >Ct4C =C,,5 C,,=D BC =D 0 C00D$8C$=Câ3Câ B
            var childMock = (Domain.UnitBase)Activator.CreateInstance(parent.GetType());

            childMock.ParentId = parent.Id;
            childMock.Parent = null;

            return Task.FromResult(childMock);
        }
    }

    public UnitServiceStandaloneTests()
    {
        // 2. Aä?Dd8C, 4Cä;Cd=D² 1D`BDÂ 4C`O TestDbContext
        _options = new DbContextOptionsBuilder<TestDbContext>()
            .UseInMemoryDatabase(Guid.NewGuid().ToString())
            .Options;

        // 3. AÄ>C=0CT< DD0C @C,,:D2 8CÄ5C0=Câ 4C`O TestDbContext
        var factoryMock = new Mock<IDbContextFactory<TestDbContext>>();

        factoryMock.Setup(f => f.CreateDbContextAsync(It.IsAny<CancellationToken>()))
            .ReturnsAsync(() => new TestDbContext(_options)); // ReturnsAsync D4?D >D"0CTB Task.FromResult

        _factory = factoryMock.Object;
    }

    // 4. A= >C0BCT:D B D$>Cd5 CD>C`6CT= C0@C,,=C,,<C BDÂ ?D 0C$8C`LCÔKCR >CôFC,,8D
    public class TestDbContext(DbContextOptions<TestDbContext> options)
        : ApplicationDbContext(options, new ConfigurationBuilder().Build())
    {
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);

            modelBuilder.Entity<Domain.UnitBase>().UseTptMappingStrategy();
            modelBuilder.Entity<TestUnit>().ToTable("TestUnits");
        }

        public override async Task<int> SaveChangesAsync(CancellationToken cancellationToken =
            default)
        {
            IEnumerable<EntityEntry<Domain.UnitBase>> entries =
                ChangeTracker.Entries<Domain.UnitBase>()
                    .Where(e => e.State is EntityState.Added or EntityState.Modified);

            foreach (EntityEntry<Domain.UnitBase> entry in entries)
            {
                Domain.UnitBase unit = entry.Entity;
                if (unit.ParentId.HasValue)
            }
        }
    }
}

```

```

        // At0C4@D46C 5CÄ @Cä4C,,BCT;Dò 4C´O Cò@Cä2CT@C¤8 (C" -äÖVö÷' f-æB @C 1CäBC 5D" 1D´AD$@Cä
        Domain.UnitBase? parent = Set<Domain.UnitBase>().Find(unit.ParentId.Value);
        if (parent != null)
        {
            // 1. Aô@C 2C,,;Cä ÷6-F-öâ ,, "CT@CÄ8C00C´LCÔKC´ Cct5C²•
            if (parent.Kind == UnitKind.Position)
                throw new InvalidOperationException("Position node cannot have
children.");
        }

        // 2. Aô@C 2C,,;Cä FW 'FÖVçB ,, CR <Cä6CTB D >CD5D 6C BDÄ ÷6-F-öâ•
        if (parent.Kind == UnitKind.Department && unit.Kind == UnitKind.Position)
            throw new InvalidOperationException($"A00D CD,,5C08CR 8CT@C @DT8Cfç >C JCT:D" :C B

'{parent.Kind}'.");
        }

        // 3. Aô@C 2C,,;Cä 8Ct>C´Odd8C, ... &öGV7F-öâö7F÷& vRöG& ç7 ÷'B =CR ACÄ5D,,8C$0DäBD O)
        if (parent.Kind is UnitKind.Production or UnitKind.Storage or
UnitKind.Transport)
        {
            if (unit.Kind != parent.Kind && unit.Kind != UnitKind.Position)
                throw new InvalidOperationException($"A00D CD,,5C08CR 8CT@C @DT8Cfç w·Væ-Bä¶-æ

'{parent.Kind}'.");
        }
    }

    }
}

return await base.SaveChangesAsync(cancellationToken);
}

[Fact]
public async Task GetByKindAsync_Should_Return_Units_Matching_Complex_Mask()
{
    // Arrange
    var service = new TestUnitService(_factory);

    // BôGCT9C¤0 C$ECä4C,,B C" <C AC¤C Storage
    await service.AddAsync(new TestUnit { Id = Guid.NewGuid(), Name = "Cell_01", Kind =
UnitKind.Storage, Type = UnitType.Cell });
    // B BC =Cä: C$ECä4C,,B C" <C AC¤C Production
    await service.AddAsync(new TestUnit { Id = Guid.NewGuid(), Name = "Lathe_01", Kind =
UnitKind.Production, Type = UnitType.MachineTool });

    // Act
    List<Domain.UnitBase> storageUnits = await service.GetByKindAsync(UnitKind.Storage);

    // Assert
    storageUnits.Should().ContainSingle(x => x.Name == "Cell_01");
    storageUnits.Should().NotContain(x => x.Name == "Lathe_01");
}

[Fact]
public async Task GetByTypeAsync_Should_Handle_Specific_Flags()
{
    // Arrange
    var service = new TestUnitService(_factory);
    await service.AddAsync(new TestUnit { Id = Guid.NewGuid(), Name = "Crane_1", Kind =
UnitKind.Storage, Type = UnitType.Crane });
    await service.AddAsync(new TestUnit { Id = Guid.NewGuid(), Name = "Vehicle_1", Kind =
UnitKind.Transport, Type = UnitType.Vehicle });

    // Act
    List<Domain.UnitBase> result = await service.GetByTypeAsync(UnitType.Crane);

    // Assert
    result.Should().ContainSingle(x => x.Name == "Crane_1");
}

[Fact]
public async Task MoveAsync_Should_Throw_When_Moving_Into_Position()
{
    // Arrange
    var service = new TestUnitService(_factory);
    var posId = Guid.NewGuid();

```

```

        var machineId = Guid.NewGuid();
    }

    await service.AddAsync(new TestUnit { Id = posId, Name = "Terminal Point", Kind =
UnitKind.Position, Type = UnitType.Other });
    await service.AddAsync(new TestUnit { Id = machineId, Name = "Mobile Machine", Kind =
UnitKind.Production, Type = UnitType.MachineTool });
    }

    // Act & Assert
    Func<Task> act = async () => await service.MoveAsync(machineId, posId);

    await act.Should().ThrowAsync<InvalidOperationException>()
        .WithMessage("Position node cannot have children.");
}

[Fact]
public async Task MoveAsync_Should_Prevent_Circular_Dependencies()
{
    // Arrange
    var service = new TestUnitService(_factory);
    var idA = Guid.NewGuid();
    var idB = Guid.NewGuid();

    // AD>C 0C$;Dô5CÂ ¶-æB 8 Type, D$0C¢ :C : Cä=C, >C 0Ct0D$5C´LCÔK (required / init)
    await service.AddAsync(new TestUnit
    {
        Id = idA,
        Name = "Node A",
        Kind = UnitKind.Department,
        Type = UnitType.Workshop
    });

    await service.AddAsync(new TestUnit
    {
        Id = idB,
        Name = "Node B",
        ParentId = idA,
        Kind = UnitKind.Production,
        Type = UnitType.Section
    });

    // Act & Assert
    Func<Task> act = async () => await service.MoveAsync(idA, idB);

    await act.Should().ThrowAsync<InvalidOperationException>()
        .WithMessage("*Bd8C¢;C,,GCTAC¢0Dò 7C 2C,,AC,,<CäAD$L*");
}

[Fact]
public async Task GetFullTreeAsync_Should_Maintain_Hierarchy_And_References()
{
    // Arrange
    var service = new TestUnitService(_factory);
    var rootId = Guid.NewGuid();
    var sectionId = Guid.NewGuid();

    await service.AddAsync(new TestUnit
    {
        Id = rootId,
        Name = "Dept",
        Kind = UnitKind.Department,
        Type = UnitType.Workshop
    });

    await service.AddAsync(new TestUnit
    {
        Id = sectionId,
        Name = "Section",
        Kind = UnitKind.Production,
        Type = UnitType.Section,
        ParentId = rootId
    });

    await service.AddAsync(new TestUnit
    {
        Id = Guid.NewGuid(),
        Name = "Workstation",

```



```

        Kind = UnitKind.Production,Ø
        Type = UnitType.Workstation,Ø
        ParentId = sectionIdØ
    });Ø
Ø
    // ActØ
    Domain.UnitBase? tree = await service.GetFullTreeAsync(rootId);Ø
Ø
    // AssertØ
    tree.Should().NotBeNull();Ø
    tree!.Children.Should().HaveCount(1);Ø
    tree.Children.First().Children.Should().HaveCount(1);Ø
}Ø
Ø
[Fact]Ø
public async Task MoveAsync_Department_Should_Not_Contain_Position()Ø
{Ø
    // ArrangeØ
    var service = new TestUnitService(_factory);Ø
    var deptId = Guid.NewGuid();Ø
    var positionId = Guid.NewGuid();Ø
Ø
    // 1. B >Ct4C 5CÂ CT?C @D$0CÄ5CÔBØ
    await service.AddAsync(new TestUnitØ
    {Ø
        Id = deptId,Ø
        Name = "Main Dept",Ø
        Kind = UnitKind.Department,Ø
        Type = UnitType.WorkshopØ
    });Ø
Ø
    // 2. B >Ct4C 5CÂ Cä7C,,FC,,N (D >C4;C ACÔ> D$2Cä5C' <C AC=5, DÔBCä <Cä6CTB C KD$L Cell)Ø
    await service.AddAsync(new TestUnitØ
    {Ø
        Id = positionId,Ø
        Name = "Work Point 1",Ø
        Kind = UnitKind.Position,Ø
        Type = UnitType.CellØ
    });Ø
Ø
    // ActØ
    // AôKD$0CT<D 0 Cö5D 5CÄ5D BC,,BDÂ ÷6—F—öâ 2 DepartmentØ
    Func<Task> act = async () => await service.MoveAsync(positionId, deptId);Ø
Ø
    // AssertØ
    // Aä6C,,4C 5CÂ >D,,8C :D2 >D" Væ—D& 6T†—W& &6‡•G&—vvW-
    await act.Should().ThrowAsync<InvalidOperationException>()Ø
        .WithMessage($"*C=0D$5C4>D 8C, u ÷6—F—öâr =CR <Cä6CTB C KD$L C$;Cä6CT= C" tFW 'FÖVçBrç""½
}Ø
Ø
[Fact]Ø
public async Task MoveAsync_Production_Should_Contain_Production_Subnode()Ø
{Ø
    // ArrangeØ
    var service = new TestUnitService(_factory);Ø
    var parentProdId = Guid.NewGuid();Ø
    var childProdId = Guid.NewGuid();Ø
Ø
    await service.AddAsync(new TestUnitØ
    {Ø
        Id = parentProdId,Ø
        Name = "Production Area",Ø
        Kind = UnitKind.Production,Ø
        Type = UnitType.WorkshopØ
    });Ø
Ø
    await service.AddAsync(new TestUnitØ
    {Ø
        Id = childProdId,Ø
        Name = "Machine Node",Ø
        Kind = UnitKind.Production,Ø
        Type = UnitType.MachineToolØ
    });Ø
Ø
    // ActØ
    Func<Task> act = async () => await service.MoveAsync(childProdId, parentProdId);Ø
}

```

```

    // Assert
    // At4CTAD 8D :C`NDt5C08D0 1D`BDÂ =CR 4Cä;Cd=CâÂ BC : C0C0 &öGV7F-öâ <Cä6CTB D >CD5D 6C BDÂ &öGV7
    await act.Should().NotThrowAsync();
}
Domain.UnitBase? updated = await service.GetByIdAsync(childProdId);
updated!.ParentId.Should().Be(parentProdId);
}
}
</file>

<file path="Promatis.Net.MES.Tests/UnitBase/AppDbContext.cs">
using Microsoft.EntityFrameworkCore;
namespace Promatis.Net.MES.Tests.UnitBase;
{
    // B$5D BCä2D`9 C0>C0BCT:D B C$=D4BD 8 C0@CäAD$@C =D BC$0 C,,<CT= D$5D BCä2D
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }
        // A,,AC0>C`LCtCCT< C 0Ct>C$KC` BC,,?, DtBCä1D² BD 8C43CT@ CÄ>C2 4CT;C BDÂ 6WCÄVæ-D& 6Sâ,•
        public DbSet<Domain.UnitBase> Units => Set<Domain.UnitBase>();
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            // 1. A00D BD 0C,,2C 5CÄ E B ,,C : C" >D =Cä2C0>CÄ ?D >CT:D$5)
            modelBuilder.Entity<Domain.UnitBase>().UseTptMappingStrategy();
            // 2. B4:C 7D`2C 5CÄÂ GD$> TestUnit – D0BCä =C AC`5CD=C,,:D
            modelBuilder.Entity<TestUnit>().ToTable("TestUnits");
        }
    }
}
</file>

<file path="Promatis.Net.MES.Tests/UnitBase/TestUnit.cs">
namespace Promatis.Net.MES.Tests.UnitBase;
{
    public class TestUnit : MES.Domain.UnitBase { }
}
</file>

<file path="Promatis.Net.MES.Tests/UnitBase/TestUnitValidator.cs">
using Promatis.Net.MES.Domain;
namespace Promatis.Net.MES.Tests.UnitBase;
{
    public class TestUnitValidator : UnitBaseValidator<TestUnit> { }
}
</file>

<file path="Promatis.Net.MES.Tests/UnitBase/UnitBaseArchitectureTests.cs">
using FluentAssertions;
using FluentValidation.TestHelper;
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Data;
using Promatis.Net.MES.Data;
using Promatis.Net.MES.Domain.Interface;
namespace Promatis.Net.MES.Tests.UnitBase;
{
    public class UnitBaseArchitectureTests
    {
        private readonly TestUnitValidator _validator = new();
        private readonly DbContextOptions<AppDbContext> _options;
        public UnitBaseArchitectureTests()
        {
            _options = new DbContextOptionsBuilder<AppDbContext>()
                .UseInMemoryDatabase(Guid.NewGuid().ToString())
                .Options;
        }
        private static EntityCancelEventArgs<MES.Domain.UnitBase> CreateArgs(MES.Domain.UnitBase
entity, DbContext context)
        {
            return new EntityCancelEventArgs<MES.Domain.UnitBase>(
entity,

```

```

        EntityStateChangeEnum.Added, Ð
        new List<PropertyChangeInfo>(), Ð
        context); Ð
    } Ð
Ð
    #region A$0C`8CD0Dd8D0 „ C AC=8 C, 1C 7Cä2D`5 C0@C 2C„;C •
Ð
    [Theory] Ð
    [InlineData(UnitKind.Storage, UnitType.Cell, true)] // B0GCT9C=0 C" !C=;C 4CR r C-
    [InlineData(UnitKind.Storage, UnitType.Workshop, false)] // Bd5DR 2 B :C`0CD5 – AäHC„1C=0 (C 8D" =CR ACä2
    [InlineData(UnitKind.Production, UnitType.Table, true)] // B BCä; C" D >C„7C$>CDAD$2CR r C-
    [InlineData(UnitKind.Position, UnitType.Other, true)] // A0@CäGCT5 C" Cä7C„FC„8 – Aä:Ð
    [InlineData(UnitKind.Position, UnitType.Table, false)] // B BCä; C" Cä7C„FC„8 – AäHC„1C=0 (D >C4;C AC0>
    public void Validator_Should_Check_Bitwise_Compatibility(UnitKind kind, UnitType type, bool
isValid) Ð
    { Ð
        var unit = new TestUnit { Kind = kind, Type = type, Name = "Test" }; Ð
        TestValidationResult<TestUnit>? result = _validator.TestValidate(unit); Ð
Ð
        if (isValid) Ð
        { Ð
            result.ShouldNotHaveAnyValidationErrors(); Ð
        } Ð
        else Ð
        { Ð
            result.ShouldHaveValidationErrorFor(x => x) Ð
                .WithErrorMessage("B$8C0 =CR ACä>D$2CTBD BC$CCTB C=0D$5C4>D 8C,â""½
        } Ð
    } Ð
Ð
    #endregion
Ð
    #region B$@C„3C45D „ D 0C$8C`0 C$;Cä6CT=C0>D BC, ¶!-æB•
Ð
    [Fact] Ð
    public async Task Trigger_Should_Allow_Root_Units() Ð
    { Ð
        await using var context = new AppDbContext(_options); Ð
        var root = new TestUnit { ParentId = null, Kind = UnitKind.Department, Type =
UnitType.Other }; Ð
Ð
        var trigger = new UnitBaseHierarchyTrigger(); Ð
        EntityCancelEventArgs<Domain.UnitBase> args = CreateArgs(root, context); Ð
Ð
        await trigger.HandleAsync(args); Ð
Ð
        args.Cancel.Should().BeFalse(); Ð
    } Ð
Ð
    [Fact] Ð
    public async Task Trigger_Should_Reject_Position_In_Department() Ð
    { Ð
        await using var context = new AppDbContext(_options); Ð
        var parentId = Guid.NewGuid(); Ð
Ð
        context.Units.Add(new TestUnit { Id = parentId, Kind = UnitKind.Department, Type =
UnitType.Other, Name = "Dept" }); Ð
        await context.SaveChangesAsync(TestContext.Current.CancellationToken); Ð
Ð
        var child = new TestUnit { ParentId = parentId, Kind = UnitKind.Position, Type =
UnitType.Other, Name = "Pos" }; Ð
        var trigger = new UnitBaseHierarchyTrigger(); Ð
        EntityCancelEventArgs<Domain.UnitBase> args = CreateArgs(child, context); Ð
Ð
        await trigger.HandleAsync(args); Ð
Ð
        args.Cancel.Should().BeTrue(); Ð
        args.ErrorMessage.Should().Contain("C05 CÄ>Cd5D" 1D`BDÂ 2C`>Cd5C0 2 'Dept' (Department)); Ð
    } Ð
Ð
    [Theory] Ð
    [InlineData(UnitKind.Production, UnitKind.Production, false)] // Bd5DR 2 Bd5DR r C-
    [InlineData(UnitKind.Production, UnitKind.Transport, true)] // B$@C =D ?Cä@D" 2 Bd5DR r C ?D 5D"5C0> (
    [InlineData(UnitKind.Storage, UnitKind.Position, false)] // A0>Ct8Dd8D0 2 B :C`0CB r C-
    [InlineData(UnitKind.Position, UnitKind.Position, true)] // A0>Ct8Dd8D0 2 A0>Ct8Dd8Dä r C ?D 5D"5C0
    public async Task Trigger_Should_Enforce_Isolation(UnitKind pKind, UnitKind cKind, bool
shouldCancel) Ð

```

```

    {
        await using var context = new AppDbContext(_options);
        var parentId = Guid.NewGuid();

        // B >Ct4C 5CÂ @Cä4C„BCT;Dò A Cò>CDECä4DöIC„< D$8Cò>Cí
        context.Units.Add(new TestUnit { Id = parentId, Kind = pKind, Type = GetFirstType(pKind),
Name = "P" });
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);

        var child = new TestUnit { ParentId = parentId, Kind = cKind, Type = GetFirstType(cKind),
Name = "C" };
        EntityCancelEventArgs<Domain.UnitBase> args = CreateArgs(child, context);
        await new UnitBaseHierarchyTrigger().HandleAsync(args);

        args.Cancel.Should().Be(shouldCancel, $"Cò>D$>CÄC DtBCâ 2C´>Cd5C08CR ¶4¶-æG0 2 {pKind} CD>C´6C0> Cò@C
    }
}
#endregion

private static UnitType GetFirstType(UnitKind kind)
    => Enum.GetValues<UnitType>().First(t => t != UnitType.None && ((int)kind & (int)t) != 0);
}
</file>

<file path="Promatis.Net.MES.UI/_Imports.razor">
@using Microsoft.AspNetCore.Components.Forms
@using Microsoft.AspNetCore.Components.Routing
@using Microsoft.AspNetCore.Components.Web
@using MudBlazor
@using Promatis.Net.UI.Components.Workspace
@using Promatis.Net.UI.Components.EditDialog
@using Promatis.Net.UI.Components.Grid
@using Promatis.Net.UI.Components.Toolbar
@using Promatis.Net.UI.Components.Tree
</file>

<file path="Promatis.Net.MES.UI/Pages/UnitOfMeasurements/Card/UnitOfMeasurementEditDialog.razor">
<EditDialog Title="@Title"
    IsNew="@IsNew"
    Model="@Model"
    Validator="@Validator"
    OnSaveAction="@OnSaveAction">
    <Tabs>
        <DialogTab Title="AäAC0>C$=Cä5" Icon="@Icons.Material.Filled.Info">
            <Content>
                <div class="d-flex flex-column gap-4 pt-2">
                    <MudTextField Label="A@C BC@>CR >C >Ct=C GCT=C„5 (A@>CB'"
                        @bind-Value="Model.Code"
                        For="@(() => Model.Code)"
                        Margin="Margin.Dense"
                        Variant="Variant.Outlined"
                        Required="true"
                        HelperText="A00C0@C„<CT@: CÄ<, Cä1/CÄ8C0Ä 2Ä HD"" óí
                    <MudTextField Label="Aô>C´=Cä5 C00C„<CT=Cä2C =C„5"
                        @bind-Value="Model.Name"
                        For="@(() => Model.Name)"
                        Margin="Margin.Dense"
                        Variant="Variant.Outlined"
                        Required="true" />
                    <MudTextField Label="Aä?C„AC =C„5 / Aô@C„<CTGC =C„5"
                        @bind-Value="Model.Description"
                        For="@(() => Model.Description)"
                        Margin="Margin.Dense"
                        Variant="Variant.Outlined"
                        Lines="3" />
                </div>
            </Content>
        </DialogTab>
    </Tabs>
</EditDialog>
</file>

```

```

<file path="Promatis.Net.MES.UI/Pages/UnitOfMeasurements/Card/UnitOfMeasurementEditDialog.razor.cs">
using Microsoft.AspNetCore.Components;
using Promatis.Net.MES.Domain;

namespace Promatis.Net.MES.UI.Pages.UnitOfMeasurements.Card;

public partial class UnitOfMeasurementEditDialog : ComponentBase
{
    [Parameter] public required string Title { get; set; }
    [Parameter] public required bool IsNew { get; set; }
    [Parameter] public required UnitOfMeasurement Model { get; set; }
    [Parameter] public required FluentValidation.Validator Validator { get; set; }
    [Parameter] public required Func<Task> OnSaveAction { get; set; }
}
</file>

```

```

<file path="Promatis.Net.MES.UI/Pages/UnitOfMeasurements/UnitOfMeasurementPage.razor">
@page "/mes/mdm/units-of-measurement"

@using Promatis.Net.MES.Domain

@inject UnitOfMeasurementPageContext Context

<CascadingValue Value="@Context" IsFixed="true">
    <WorkspacePage>
        <TopContent>
            <UiToolbar TEntity="UnitOfMeasurement"
                ActionContext="@Context"
                OnCreateTriggered="@Context.OnCreateActionAsync"
                OnEditTriggered="@Context.OnUpdateActionAsync"
                OnDeleteTriggered="@Context.OnDeleteActionAsync" />
        </TopContent>
        <BodyContent>
            @* Bt8D BC 0 CD5Cq;C @C FC,,0: GridPage C 2D$>CÄ0D$8Dt5D :C, 7C 1CT@CTB CD0CÔ=D´5 C,,7 Context.Data
            <GridPage TEntity="UnitOfMeasurement">
                <ColumnsContent>
                    <PropertyColumn T="UnitOfMeasurement" TProperty="string" Property="x => x.Code"
                        Title="A=>CB 5CD8C08DdK C,,7CÄ5D 5C08D0" †V FW%7G-ÆS0'v-GFf¢ S f²" 6Æ 73
                    <PropertyColumn T="UnitOfMeasurement" TProperty="string" Property="x => x.Name"
                        Title="A00C,,<CT=Cä2C =C,,5 CT4. C,,7CÄâ" †V FW%7G-ÆS0'v-GFf¢ 3 f²" óí
                    <PropertyColumn T="UnitOfMeasurement" TProperty="string" Property="x =>
x.Description"
                        Title="Aä?C,,AC =C,,5 / A0@C,,<CTGC =C,,5" />
                </ColumnsContent>
            </GridPage>
        </BodyContent>
    </WorkspacePage>
</CascadingValue>
</file>

```

```

<file path="Promatis.Net.MES.UI/Pages/UnitOfMeasurements/UnitOfMeasurementPageContext.cs">
using MudBlazor;
using Promatis.Net.Domain.Interface;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Service;
using Promatis.Net.MES.UI.Pages.UnitOfMeasurements.Card;
using Promatis.Net.UI.Components.Grid;

namespace Promatis.Net.MES.UI.Pages.UnitOfMeasurements;

public class UnitOfMeasurementPageContext : GridActionContext<UnitOfMeasurement>
{
    private readonly IUnitOfMeasurementService _uomService;
    private readonly IDialogService _dialogService;
    private readonly IEntityCloner _entityCloner;

    public UnitOfMeasurementPageContext(
        IUnitOfMeasurementService uomService,
        IDialogService dialogService,
        IEntityCloner entityCloner)
    {
        _uomService = uomService ?? throw new ArgumentNullException(nameof(uomService));
        _dialogService = dialogService ?? throw new ArgumentNullException(nameof(dialogService));
    }
}

```

```

        _entityCloner = entityCloner ?? throw new ArgumentNullException(nameof(entityCloner));
    }

    PageTitle = "B ?D 0C$>Dt=C,,: CT4C,,=C,,F C,,7CÄ5D 5C08Dò#½

    // A$:C`NDt0CT< C00D$8C$=D´9 Aä B20@CT6C,,< CD;Dò ?D 8C$0Ct:C, : GridPage
    DataBroker.ConfigureInMemoryMode();
}

/// <summary>
/// B$ Bt Bt Aä AD A A$ AT A,, : AÄ5D$>CB ?CT@C$8Dt=Cä3Câ =C ?Cä;C05C08Dò At#-CMD,,0 C @Cä:CT@C 4C =C0
/// </summary>
public async Task InitializeInMemoryDataAsync()
{
    try
    {
        List<UnitOfMeasurement> data = await _uomService.GetAllAsync();
        DataBroker.InMemoryItems = data;
    }
    catch (Exception ex)
    {
        Console.WriteLine($"[ERROR] AäHC,,1C00 C,,=C,,FC,,0C´8Ct0Dd8C, At#-CMD,,0 CT4C,,=C,,F C,,7CÄ5D 5C08Dóç
    }
}

// =====
// A0 AÄ A0 B² A AT A, A0!B$ B4 AT B$ A" „5%TB•
// =====

public async Task OnCreateActionAsync()
{
    var newUom = new UnitOfMeasurement();
    await OpenEditDialogAsync<UnitOfMeasurementEditDialog>(
        newUom,
        "AD>C 0C$;CT=C,,5 CT4C,,=C,,FD² 8Ct<CT@CT=C,,0",
        isNew: true,
        saveDelegate: () => _uomService.AddAsync(newUom)
    );
}

public async Task OnUpdateActionAsync(UnitOfMeasurement? row)
{
    if (row == null) return;
    UnitOfMeasurement targetClone = _entityCloner.CloneEntity(row);
    await OpenEditDialogAsync<UnitOfMeasurementEditDialog>(
        targetClone,
        $"B 5CD0C0BC,,@Cä2C =C,,5: {row.Code}",
        isNew: false,
        saveDelegate: () => _uomService.UpdateAsync(targetClone)
    );
}

public async Task OnDeleteActionAsync()
{
    if (SelectedItem == null) return;

    bool? confirm = await _dialogService.ShowDialogAsync(
        "B44C ;CT=C,,5 Ct0C08D 8",
        $"A$K CD5C´AD$2C,,BCT;DÄ=Câ ECäBC,,BCR CCD0C´8D$L CT4C,,=C,,FD2 8Ct<CT@CT=C,,0 '{SelectedItem.Name}' (
        yesText: "B44C ;C,,BDÄ"Ä 6 æ6VÄFW†Cç $ D$<CT=C ""½

    if (confirm == true)
    {
        await _uomService.DeleteAsync(SelectedItem.Id);
    }
}
}
</file>

<file path="Promatis.Net.MES.UI/Promatis.Net.MES.UI.csproj">
<Project Sdk="Microsoft.NET.Sdk.Razor">
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <Nullable>enable</Nullable>
        <ImplicitUsings>enable</ImplicitUsings>

```

```

    </PropertyGroup>
  </ItemGroup>
  <ItemGroup>
    <Compile Remove="UnitBase\*" />
    <Content Remove="UnitBase\*" />
    <EmbeddedResource Remove="UnitBase\*" />
    <None Remove="UnitBase\*" />
  </ItemGroup>
  <ItemGroup>
    <SupportedPlatform Include="browser" />
  </ItemGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.AspNetCore.Components.Web" Version="10.0.8" />
    <PackageReference Include="MudBlazor" Version="9.4.0" />
  </ItemGroup>
  <ItemGroup>
    <ProjectReference Include="..\..\Core\Configuration.Web\Promatis.Net.Configuration.Web.csproj" /
  >
    <ProjectReference Include="..\..\Core\Promatis.Net.UI\Promatis.Net.UI.csproj" />
    <ProjectReference Include="..\Promatis.Net.MES.Service\Promatis.Net.MES.Service.csproj" />
  </ItemGroup>
</Project>
</file>

<file path="Promatis.Net.MES.UI\UiModule.cs">
using MudBlazor;
using Promatis.Net.UI;
namespace Promatis.Net.MES.UI;
public class UiModule : IUiModule
{
    public string Name => GetType().Assembly.GetName().Name ?? "Unknown.Module";

    public IEnumerable<(string Title, string Href, string Icon, string? Group)> GetMenuItems()
    {
        return new List<(string Title, string Href, string Icon, string? Group)>
        {
            // Aö;CäAC¸8C' ACô@C 2CäGCÔ8C¢ BCTECÔ>C´>C48Dt5D :C„E C00D 0CÄ5D$@Cä2D
            ("AT4C„=C„FD² 8Ct<CT@CT=C„0", "/mes/mdm/units-of-measurement", Icons.Material.Filled.SettingsInput);
        };
    }
}
</file>

<file path="Promatis.Net.MES.UI/wwwroot/exampleJsInterop.js">
// This is a JavaScript module that is loaded on demand. It can export any number of
// functions, and may import other JavaScript modules if required.
export function showPrompt(message) {
    return prompt(message, 'Type anything here');
}
</file>
</files>

```